

**Е. Н. Королев, Б. Н. Тишуков,  
А. В. Мандрыкин**

# **АДМИНИСТРИРОВАНИЕ СУБД**

**Учебное пособие**



**Воронеж 2021**

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное  
учреждение высшего образования

«Воронежский государственный технический университет»

Е. Н. Королев, Б. Н. Тишуков, А. В. Мандрыкин

## **АДМИНИСТРИРОВАНИЕ СУБД**

Учебное пособие

Воронеж 2021

УДК 004.65

ББК 32.97

К682

***Рецензенты:***

*кафедра математики, информационных систем и технологий,*

*Воронежского филиала ФГБОУ ВО «ГУМРФ*

*им. адмирала С. О. Макарова»*

*(зав. кафедрой д-р техн. наук, профессор М. Л. Лапина);*

*А. П. Преображенский, д-р техн. наук, профессор Воронежского  
института высоких технологий*

**Королев, Е. Н.**

Администрирование СУБД: учебное пособие/  
К682 Е. Н. Королев, Б. Н. Тишуков, А. В. Мандрыкин; ФГБОУ ВО  
«Воронежский государственный технический университет». –  
Воронеж: Изд-во ВГТУ, 2021. – 156 с.

ISBN 978-5-7731-0947-1

Учебное пособие содержит материал по разделам администрирования систем управления базами данных. Теоретический материал сопровождается примерами программного кода, что предназначено для овладения студентами практическими навыками решения задач по администрированию СУБД.

Издание предназначено для студентов, обучающихся по направлению 09.03.02 «Информационные системы и технологии», при изучении дисциплины «Администрирование СУБД».

Ил. 6. Библиогр.: 4 назв.

**УДК 004.65**

**ББК 32.97**

*Печатается по решению редакционно-издательского совета  
Воронежского государственного технического университета*

ISBN 978-5-7731-0947-1

© Королев Е. Н., Тишуков Б. Н.,  
Мандрыкин А. В., 2021

© ФГБОУ ВО «Воронежский  
государственный технический  
университет», 2021

## ВВЕДЕНИЕ

Любое программное обеспечение не может быть представлено без использования и хранения данных. Для реализации хранения, обновления и удаления используют системы управления базами данных (СУБД). СУБД – это комплекс программных и лингвистических средств, применяемых для управления всеми данными.

К основным функциям СУБД относятся:

- возможность управления внешней памятью;
- возможность управлять оперативной памятью;
- возможность восстановить данные из БД с помощью журнала изменений;
- поддержка языков БД (DDL, DML, DCL).

MySQL – это программа для работы с СУБД. Использование этого ПО становится проще с развитием СУБД. Свою популярность она получила благодаря своему интенсивному развитию и простоте. Конечно же это в основном относится к тем людям, которые не думали быть программистами, но знания в этой области никогда не будут лишними. Если человек планирует стать администратором, то ему необходимо научиться грамотно использовать систему и знать, какие действия следует применять при возникновении тех или иных ошибок. О всех этих знаниях будет рассказано в этом учебном пособии.

В данном пособии будут рассмотрены вопросы, которые касаются администрирования. В пособии также есть список общих обязанностей администратора и инструкции по их выполнению.

Если человек, ранее работавший администратором, рассмотрит администрирование MySQL, то он может найти большое количество схожих деталей, но, конечно же, у MySQL, как и у любой другой системы, будут свои нюансы, которые необходимо знать, и которые будут рассмотрены в представленном учебном пособии.

В пособии разобраны:

1. Структуры данных для администрирования Mysql.
2. Служебные файлы для настройки СУБД.
3. Различные утилиты, к которым можно отнести mysql, mysqladmin, isamchk, myisamchk, mysqldump.
4. Вопросы безопасности mysql.
5. Особенности инсталляции.
6. Популярные средства администрирования.
7. Утилиты мониторинга.
8. Типы таблиц mysql.
9. Управление учетными записями.
10. Способы резервного копирования.
11. Команды администрирования баз данных и файлы журналов.

## 1. КОМПОНЕНТЫ СУБД MYSQL

В СУБД входит множество компонент и знание этих компонентов, заметно упростит работу не только самой СУБД, но и пользователей, которые будут работать с ней. К таким компонентам следует отнести три основных:

- сервер MySQL;
- клиентские программы и утилиты MySQL;
- каталог данных MySQL.

Программа MySQLID является основой, сервером СУБД. Ее применяют для запуска, перезагрузки и остановки сервера.

Также, существует не мало и других программ, выполняющих эту работу, и у всех есть свои достоинства и недостатки. Клиентскими программами являются различные программы MySQL, к самым важным из которых относятся:

- mysql это интерактивная программа, работающая с SQL скриптом. Она позволяет отправлять запросы SQL на сервер, получать ответы и предоставлять информацию пользователю.

- mysqladmin – программа выполняющая такие функции, как завершение работы сервера, создание и удаление баз данных. Она применяется для мониторинга работы сервера при возникновении различного рода вопросов ошибок в течении компиляции.

- isamchk или myisamchk – утилиты для анализа работы системы и ее компиляции. Если в процессе работы сервера произошли ошибки, данная программа также поможет их исправить.

- mysqldump – это средство для резервирования баз данных или их копирования на другой сервер.

Комплекс задач администрирования можно выполнить только с помощью утилиты командной строки MySQL.

Существует специальный каталог данных MySQL, использующийся сервером для хранения баз данных и файлов состояния. Если администратор или пользователь будет понимать структуру системы MySQL, то у него появится воз-

возможность узнать, как система хранит все файлы внутри системы, что для этого используется и как с учетом этих данных будет лучше оптимизировать работу отладчика. Также важно уметь работать с дисковым пространством, во избежание заполнения всей памяти и дальнейшего сбоя системы.

Как правило администратор обеспечивает работу с именем `mysql`, а именно контролирует доступ пользователей, создает новых и удаляет не нужных. К самым важным задачам общего администрирования `MySQL` относятся:

- умение включать и отключать сервер, как вручную, так и настроить на автоматический запуск и остановку. Управление сервером из командной строки, а также восстановление его работы или неисправного функционирования.

- Работа с учетными записями, настройка и управление. А также четкое понимание различия учетных записей `mysql` и `windows`.

- задача поддержки регистрационных файлов `MySQL`. В обязанности администратора входят знания регистрационных файлов, какие файлы нужны и какие следует поддерживать исправном состоянии, а какие нет. Просмотр, порядковый позволит проверить какие, нужны файлы, а какие нет.

- Работа резервирования и восстановления БД. Эта составляющая является наиболее обязательности, так как данные, пришедшие в БД, претерпевают, изменения постоянно вследствие чего могут возникнуть множество ошибок, которые приведут к сбою, вследствие чего потребуется восстановление БД.

Резервирование БД отличается от резервирования системы и незнание этого может привести к серьезным последствиям. Если файлы были зарезервированы давно, то полное восстановление в случае сбоя будет невозможно. Но возможно то, что файлы будут откатаны до момента создания резервной копии.

Бывает, появляется потребность переноса всей БД на другую машину, и в таком случае, необходимо полное копирование данных. Но простого переноса файлов на запоминающее устройство может быть недостаточно, потому что могут

понадобиться настройки самой системы, которые не будут прописаны в копируемых файлах. Администратор MySQL должен уметь выполнять эту процедуру.

- задачи настройки сервера. В любом случае, это является ясно, что люди, а именно пользователи системы, хотят, чтобы доступ к данным и работа с данными была максимально быстрой. Для этого они ставят более мощные машины, обновляют аппаратуру что требует больших затрат, но этот способ не экономичный, но для применения подобных методов можно и не разбираться в работе сервера. Администратору нужно знать лишь необходимые параметры для успешного подбора параметров машины. Но этот принцип можно заменить более экономично. Большая часть клиентов использует запросы на выборку вследствие чего, при правильной настройке сервера, улучшение показателей машины может стать не обязательной процедурой, но получить результат не хуже. Поэтому решение об изменении значений каких-либо параметров часто определяется типом запросов, отправляемых к СУБД.

- задача установки нескольких серверов. Такие задачи как правило используют люди, если хотят протестировать работу новой машины или системы, а именно то, как они взаимодействуют друг с другом. Появляются ли конфликты и др.

- задача обновления MySQL. В связи с тем, что программное обеспечение ВСЕГДА имеет динамический характер, то оно подвергается большим изменениям. Поэтому администратор должен уметь эти изменения применять. А именно обновлять ПО, а также разбираться, имеет ли смысл ставить эту систему, если она, например, бета.

## **2. БЕЗОПАСНОСТЬ MYSQL**

После установки, а также во время ее, администратор должен, обязательно заботиться о безопасности системы. А именно даже уже на этот момент ему необходимо разграничивать доступ к данным и выполнять операции:

- защита файловой системы;

- обеспечение защиты сервера;
- обеспечение восстановления системы после сбоя.

Администратор должен знать, как отлаживать и восстановить таблицы баз данных после сбоя системы.

Есть несколько программ, которые можно считать исключительно полезными при выполнении задач при администрировании СУБД MySQL. К таким программам относятся:

- программа `mysqladmin`, позволяющая выполнять всевозможные административные функции MySQL;
- программы `safemysqld` и `mysqldserver`, используемые для запуска MySQL-сервера `mysqld`;
- программа `mysqldump`, используемая для копирования и резервирования баз данных;
- утилиты `myisamchk` и `isamchk`, используемые для проверки целостности данных таблиц MySQL и операций отладки.

Защита новой установки MySQL является важной функцией.

Во время установки MySQL важно установить пароль для пользователя `root` в MySQL, по той причине, что сразу после установки права сервера не будут защищены. Предположим, что каталог данных и база данных `mysql` с таблицей решений были предварительно инициализированы. Для выполнения их инициализации достаточно выполнить сценарий `mysql_install_db` на компьютерах с ОС UNIX. Если компьютер использует ОС Windows, то каталог данных и база данных `mysql` необходимо будет проинициализировать, запустив программу `Setup`.

После первой установки MySQL привилегии в таблице разрешений базы данных `mysql` на компьютере устанавливаются так:

— Сначала нужно войти в систему как основной пользователь `root` с локального компьютера, можно без пароля. Пользователь `root` обладает всеми возможными правами (в них входят и административные), может выполнять любые операции.

— Затем права анонимного доступа предоставляются тем пользователям, которые подключаются с локального компьютера к базе данных test или какой-либо другой базе данных, название которой начинается со слова test. Пользователи, зашедшие анонимно, могут выполнять любые операции с такими таблицами, но они не обладают привилегиями администратора. В качестве соединения с сервером с локального компьютера можно определить, как имя главного компьютера localhost, так и его реальное имя. Например, если сервер располагается на компьютере pit-viper.snake.net, то пользователь этого компьютера сможет подключиться без пароля к серверу для работы с базой данных test используя одну из двух команд:

- % mysql -h localhost test;
- % mysql -h pit-viper.snake.net test;

Поскольку после исходной установки подключиться к серверу MySQL можно в качестве пользователя root совершенно без пароля, то первой основной задачей любого администратора MySQL является установка пароля для пользователя root. После этого, в зависимости от того, как был установлен пароль, может потребоваться указать серверу перезагрузить таблицы разрешений, чтобы загрузить в память все сделанные изменения.

Новый пароль для root можно установить выполняя команду:

```
mysqladmin -u root password "my password"
```

Для операций такого рода также можно использовать программу mysql и, следовательно, обновить таблицу разрешений в базе данных mysql:

```
mysql> -u root mysql
mysql> UPDATE user SET Password=PASSWORD("my
pass") WHERE User="root".
```

Команда mysql и оператор update применяются в старых версиях MySQL и в бесплатно распространяемых версиях для Windows.

После установки пароля, важно решить, нужно ли серверу перезагружать таблицы разрешений. Чтобы это выполнить, надо использовать команду:

```
% mysqladmin -u root status.
```

Если сервер подключен с использованием пользователя root без пароля, то следует незамедлительно перезагрузить таблицы, введя следующую команду:

```
% mysqladmin -u root reload.
```

Следующая основная обязанность администратора MySQL состоит в том, чтобы обеспечить согласованный и продолжительный процесс работы сервера, который позволит пользователям получать к нему доступ в любое удобное время. Иногда возникает необходимость временно приостановить работу сервера MySQL. Необходимость такого рода может возникнуть при перемещении базы данных. В тоже время необходимо быть уверенным, что сервер не выполняет никаких обновлений в базе данных.

Все инструкции, представленные в учебном пособии, применяются исключительно к операционным системам семейства UNIX.

Рассмотрим запуск сервера MySQL непривилегированным пользователем.

Сервер имеет возможность запускаться двумя способами – это вручную и автоматически. В первом случае сервер запускается от имени пользователя, под которым зарегистрирован администратор, который запускает сервер. То есть, если имя администратора (петя)admin, и он запускает сервер, то сервер продолжит работу с правами пользователя (петя)admin. Если администратор авторизуется под пользователем root, используя команду su, то сервер запустится с правами пользователя root.

Но все время вручную запускать сервер очень неудобно. Для этого процесса лучше настроить его автоматический запуск во время загрузки системы. Процесс запуска на компьютерах под управлением UNIX выполняется системой с использованием пользователя root, поэтому, те процедуры, ко-

торые запускаются во время этого процесса, продолжают работать с привилегиями пользователя root.

Администратору нужно помнить и знать о двух рекомендациях, которыми следует руководствоваться при настройке процедуры запуска сервера MySQL:

- Следует настроить сервер так, чтобы у него не было привилегий root. Часто рекомендуется наложить ограничения на возможности тех процессов, которые не требуют прав доступа root. Эти права не требуются и демону `mysqld`.

- Следует настроить сервер так, чтобы он все время работал, используя одного пользователя. Менять имена пользователей при запуске сервера не рекомендуется, так как в этом случае файлы и каталоги с данными будут создаваться разными владельцами. Иногда это приводит к невозможности получения доступа к базам данных или таблицам. Чтобы избежать проблемы такого рода, следует запускать сервер от имени одного пользователя.

Необходимо выполнить следующие шаги, чтобы запустить сервер как обычный пользователь, не имеющий широких прав:

- выбрать учетную запись, которая предназначена для запуска сервера. Демон `mysqld` имеет возможность работать от имени любого пользователя, но в большинстве случаев для него лучше создать отдельную учетную запись. Также, для работы с MySQL можно создать специальную группу. Например, так пользователь и группа могут иметь имена `mysqladm` и `mysqlgrp`;

- завершить работу сервера (если он запущен);
- изменить права доступа к каталогу данных, а также для всех его подкаталогов и файлов для того, чтобы новым владельцем этих элементов был пользователь `mysqladm`. (команда изменения владельца `chown`);

- изменить полномочия доступа к каталогу данных и всем его подкаталогам и файлам, для того чтобы с ними мог работать только пользователь `mysqladm`. Наиболее эффективная мера предосторожности – запретить доступ к

данным всем остальным пользователям (команда изменения прав доступа `chmod`).

Ставя права доступа и режим для каталога данных и его содержимого, необходимо узнать символические связи. Для этого, необходимо обязательно перейти в каталоги, на которые указывают эти связи, после чего права доступа изменить в соответствии с их содержимого. На данном этапе бывает такое, что возникают проблемы, если содержащий все связанные файлы каталог не принадлежит владельцу каталога данных. Способ решения таких проблем один — это авторизоваться в качестве пользователя `root`.

После этого, необходимо убедиться в верном запуске сервера, предварительно авторизовавшись в качестве пользователя `mysqladm`.

Существует несколько способов запустить сервер. Перейти к выбору способа запуска сервера можно после определения учетной записи, используемой для его работы. Сервер может быть запущен вручную из командной строки или автоматически в процессе загрузки системы. Есть три основных способа запуска сервера:

- Непосредственный вызов `mysqld`. Этот способ, возможно, самый распространенный.

- Вызов сценария `safemysqld`. В ходе своей работы этот сценарий пытается найти программу сервера `mysqld` и каталога данных, после чего запускает сервер с соответствующими параметрами. Сценарий `safemysqld` сохраняет все сообщения об ошибках сервера в специальный файл ошибок, который располагается в каталоге данных. Кроме этого, `safemysqld` контролирует работу сервера и в случае сбоя перезагружает его. Такой сценарий часто применяется в BSD-версиях UNIX.

- Вызов сценария `mysql.server`. Этот сценарий запускает сервер посредством запуска сценария `safemysqld`. Такой сценарий предназначен для использования на компьютерах с системой запуска/завершения работы в семействе System V (Red Hat). Такая система включает несколько каталогов со сценариями, которые вызываются при входе или выходе с

определенного уровня работы. С помощью аргументов start и stop есть возможность определить, что следует делать дальше: запустить сервер или остановить его работу.

### **3. ПОПУЛЯРНЫЕ УТИЛИТЫ И СРЕДСТВА АДМИНИСТРИРОВАНИЯ MYSQL**

#### **3.1. MySQL Workbench**

Среда MySQL Workbench от компании Oracle для Linux и Windows – это мощная среда с большими возможностями, которая позволяет создавать базы данных, управлять ими и визуально проектировать. Все инструменты являются доступными для настройки серверов, резервного копирования и восстановления, администрирования учетных записей, аудита и простого мониторинга состояния. Это также позволяет легко мигрировать с различных других СУБД — PostgreSQL, MS SQL Server, Sybase ASE и прочих. Используя дополнительные плагины, возможности данной среды также могут быть расширены.

Workbench делают популярным, следующие возможности:

- представление модели бд в графическом виде;
- наличие простого механизма для создания связей;
- функции, представляющие возможность восстанавливать структуры таблиц и связей;
- возможность использования редактора sql-запросов.

#### **3.2. MySQL PHPMyAdmin**

PHPMyAdmin — это приложение для администрирования СУБД MySQL. PHPMyAdmin – представляет Web-интерфейс, с помощью которого можно просматривать содержимое БД, вносить изменения и редактирования объектов БД.

PHPMyAdmin дает возможность выполнить большинство операций в интуитивно понятной среде для управления базами данных, индексами, работе с таблицами, настройку

репликации, правами доступа, экспорт информации, просматривать статистику, резервное копирование или восстановление, и так далее. В то же время имеется ввод любых непосредственных запросов SQL. Также, данная среда имеет возможность поддержки управления нескольких серверов. Все процессы интуитивно понятны, и пользователь может справиться с администрированием без какого-либо специального обучения, с любого устройства с браузером.

PHPMyAdmin делают популярным следующие функции:

- управление СУБД MySQL без непосредственного ввода команд SQL;
- интенсивное развитие;
- PHPMyAdmin, в качестве панели управления, дает возможность администрирования выделенных БД;
- интеграция PHPMyAdmin в личные разработки с помощью лицензии GNU General Public License и др.

### **3.3. Утилиты мониторинга**

Данная СУБД, как и любое другое приложение, требует постоянного мониторинга своей работы для того, чтобы в случае возникновения сбоев или любых других проблем, была возможность легко найти узкое место. Общая информация о работе MySQL может быть получена стандартным клиентом `mysqladmin`. Но такие запросы, как `SHOW VARIABLES`, `SHOW QUERY LOG`, `SHOW GLOBAL STATUS`, `SHOW PROCESSLIST` и другие редко дают четкую картину, поскольку всегда есть медленные запросы, которые не обязательно могут влиять на работу службы. Существует также утилита `mysqldumpslow`, анализирующая данные `slow.log` и отражающая частые медленные запросы.

Enterprise предоставляет специальный инструмент MySQL Enterprise Monitor, который в режиме реального времени выдает информацию о производительности и доступности всех баз данных среды MySQL. Кроме того, специальные плагины доступны для большинства систем мониторинга,

включая Cacti, Zabbix, Open Source Nagios, Ganglia, Все плагины должны быть правильно настроены, для верного представления информации о том, что происходит в MySQL. Когда задача стоит только в контроле СУБД, то е всегда является рациональным разворачивать полноценную систему мониторинга. В таком случае лучше использовать специализированные инструменты.

Аудит MySQL является одной из главных задач. А неправильная настройка эксплуатации любого программного средства – это одна из основных проблем. Как правило, после того, как среда установлена, возникает ряд проблем, таких как присутствуют лишние учетные записи, пользователи, ставящие пароли, делают их слишком простыми, которые в дальнейшем не сложно подобрать, также лишними могут присутствовать и тестовые базы. Такие проблемы решаются лишь при применении постоянного аудита, который также будет необходим, если обрабатываются персональные данные или конфиденциальная информация.

В таком случае, следует образовать среду, которая будет соответствовать определенным стандартам безопасности (SOX, HIPAA и прочим), при устранении неполадок и расследовании инцидентов. Как только программа закончит свою установку, необходимо применять `mysql_secure_installation`. Она позволяет выполнять минимальный набор проверок, которые способны скорректировать общие настройки безопасности.

Следующей главной задачей является тюнинг MySQL. Очень тонким процессом является оптимизация настроек. Необходимо, используя собранную статистику изменить только те настройки, которые реально смогут повлиять на производительность системы. Одним из популярных инструментов для среды MySQL является Perl-скрипт MySQLTuner. К нему есть доступ в репозиториях многих дистрибутивов Linux. Он может считывать настройки сервера в настоящий момент и устанавливать MySQL, затем предлагает рекомендации их изменения.

### 3.4. Файл параметров MY.INI

Считывание параметров по умолчанию в среде MySQL операционной системы Windows, выполняется из файла my.ini. В этом файле все данные разбиты на 3 раздела:

- Первый раздел – [client]
- Второй раздел – [mysql]
- Третий раздел – [mysqld]

В первом разделе расположены настройки по умолчанию для всех клиентов сервера MySQL.

Во втором разделе расположены настройки программы MySQL.

В третьем разделе расположены настройки сервера MySQL.

В файле my.ini возможны еще два раздела. Это раздел [mysqldump]. Он создан для настроек утилиты mysqldump. И это раздел [myisamchk]. Он нужен для настроек утилиты myisamchk.

Параметры, разделов client и mysqld представлены в таблице.

Таблица

| Параметр                                               | Комментарий                                                                 |
|--------------------------------------------------------|-----------------------------------------------------------------------------|
| Параметры для раздела [client]:                        |                                                                             |
| character-sets-dir=/usr/local/mysql-5.5/share/charsets | Путь к папке, где находятся таблицы перекодировок                           |
| port = 3306                                            | Порт                                                                        |
| default-character-set = cp1251                         | Кодировка консоли mysql.exe по умолчанию                                    |
| socket = /tmp/mysql5.sock                              | Файловый сокет, возможность подключения консольным клиентом                 |
| Параметры для раздела [mysqld]:                        |                                                                             |
| character-set-server = utf8                            | Серверная кодировка таблиц                                                  |
| bind-address = 127.0.0.1                               | Обработка запросов с локальной машины                                       |
| user = mysql                                           | Пользователь операционной системы, под которым авторизовывается база данных |

Также в файле `my.ini` располагаются:

- каталог, где хранится база данных
- каталог, где хранятся временные файлы
- тип хранилища таблиц по умолчанию
- ограничение на максимальное число соединений с

БД и т.д., включая опции на различные типы таблиц.

Например, опции `mysqld - InnoDB`:

- `innodb_log_group_home_dir` - путь к каталогу под журнал транзакций
- `innodb_buffer_pool_size` - размер буфера под все нужды
- `innodb_log_files_in_group` - количество файлов журналов транзакций в группе журналов
- `innodb_log_file_size` - размер каждого файла журнала в группе журналов
- `innodb_log_buffer_size` - размер буфера, который в InnoDB используется для записи информации файлов журналов на диск.

После внесения всех изменений в файл `my.ini`, необходимо, чтобы сервер MySQL был перезагружен.

Пример, того, как выглядит файла `my.ini`:

```
[client]
```

```
port = 3306
```

```
socket = /tmp/mysql5.sock
```

```
character-sets-dir = /usr/local/mysql-5.5/share/charsets
```

```
# Кодировка консоли mysql.exe по умолчанию.
```

```
default-character-set = cp1251
```

```
# The MySQL server
```

```
[mysqld]
```

```
# Обработать запросы только с локальной машины  
(повышает безопасность).
```

```
# Если вы хотите, чтобы MySQL-сервер был доступен  
из сети, уберите следующую ниже директиву.
```

```
bind-address = 127.0.0.1
```

```
port = 3306
# Внутренняя кодировка таблиц на сервере.
character-set-server = utf8
# Кодировка клиента по умолчанию (преобразование из
cp1251 в utf8 и обратно происходит автоматически).
init-connect = "set names cp1251"
tmpdir = /tmp
socket = /tmp/mysql.sock
skip-external-locking
key_buffer_size = 16K
max_allowed_packet = 1M
table_open_cache = 4
sort_buffer_size = 64K
read_buffer_size = 256K
read_rnd_buffer_size = 256K
net_buffer_length = 2K
thread_stack = 128K
```

#### **4. ТИПЫ ТАБЛИЦ MYSQL**

Старая версия MySQL 3.23 поддерживала 3 формата: ISAM, HEAP и MyISAM – без поддержки транзакций.

В новых версиях были добавлены еще несколько видов, в частности это InnoDB или BDB с поддержкой транзакций. Какая из двух вариантов будет стоять, на машине, зависит от варианта установки.

Преимущества таблиц с транзакциями:

- Такие таблицы наиболее надежные. В случае сбоя системы, в процессе работы, есть возможность автоматического восстановления.
- Такие таблицы также могут сочетать в себе несколько операторов. После чего подтверждать все изменения командой COMMIT.
- Такие таблицы имеют возможность откатить измененные данные.
- Возможность восстановления всех внесенных изменений, если в процессе работы произошел сбой.

В среде MySQL пользователь в процессе работы имеет возможность изменить тип таблиц.

В MyISAM есть полезная возможность по сжатию данных. MYD (MYData), расширением файла с индексами является. MYI (MYIndex). У таблиц MyISAM, есть возможность проверки и восстановления используя утилиту `myisamchk`.

Таблицы с типом MyISAM как правило, используются в интернет страницах, где нет жестких критериев к надежности данных и используется выборка.

Таблицы MyISAM используются на различных платформах, что делает их платформенно-независимыми. Файлы таблиц могут быть перемещены без всякого преобразования между компьютерами различных архитектур и имеющих разные операционные системы.

Файлы с расширением .MYD (MYData) созданы для того, чтобы хранить в них данные. Индексные файлы имеют расширение .MYI (MYIndex), а файлы с расширением .frm — содержат в себе схему таблицы. Редко, но случается такое, что индексные файлы портятся, удаляются и приходят в неисправное состояние, тогда система автоматически генерирует индексные файлы из файла .frm.

В таблицах MyISAM записи могут быть в 3 форматах:

- Динамические.
- Сжатые.
- Фиксированные по длине.

К недостаткам таких таблиц можно отнести следующие пункты:

- Транзакции и внешние ключи отсутствуют.
- Нет автоматического восстановления.
- Отсутствие средств резервного копирования. Предлагаемая для создания резервных копий утилита `mysqldump` является не средством резервного копирования, а средством экспорта в текст (то есть она образует последовательность операторов INSERT, которые в свою очередь восстанавливают содержимое таблицы). Для реализации сохранения целостности БД `mysqldump` блокирует таблицы, следствием чего яв-

ляется полная остановка работы системы на время ее исполнения.

- Потребность в периодическом выполнении команды ANALYZE для того, чтобы улучшить работу оптимизатора запросов.

- При использовании предложения ORDER BY языка SQL используется слабая реализация сортировки.

Представленные недостатки можно заметить при высокой нагрузке: более 400 клиентов, которые будут выполнять сложные запросы по БД размером 2-3 ГБ.

В настоящее время в среде MySQL пока еще есть тип таблиц ISAM.

Но в процессе такой тип таблиц выйдет из оборота, так как на его место приходит более обновленная версия.

Есть еще такой тип таблиц как HEAP. Такой формат использует хэш-индексы. Данный вид таблиц может храниться только в памяти. Благодаря этому, запросы к таким таблицам обрабатываются очень быстро. Но такие таблицы в случае сбоя не будут подлежать восстановлению, и данные будут утеряны. В большинстве случаев такой тип таблиц используется для реализации временных таблиц.

Если пользователь использует таблицы этого типа ему необходимо обращать внимание на:

- Явное указание такого параметра, как MAX\_ROWS в операторе CREATE, чтобы в случае большого запроса не была занята вся память.

- Наличие необходимой памяти для создаваемого запроса

```
CREATE TABLE t1 (id INT, param1 VARCHAR2(40) NOT NULL) TYPE MYISAM;
```

```
CREATE TABLE t2 (id INT, param1 VARCHAR2(240) NOT NULL) TYPE=HEAP;
```

Таблицы с типом InnoDB входит во все стандартные сборки для различных операционных систем. Наличие внешних ключей и механизма транзакций является главной отличительной чертой таблиц с типом InnoDB от других подсистем низкого уровня среды MySQL.

В середине 2001 года появилась поддержка типа таблиц InnoDB. По умолчанию основным хранилищем начинается с версии 5.5.

Данные InnoDB, в отличие от таблиц MyISAM, в которых для всех таблиц создается один файл данных, по умолчанию хранятся в больших общих используемых файлах (это можно изменить в настройках опций `innodb_file_per_table`), что дает возможность использовать постраничный кэш страниц БД. Надежное хранение данных обеспечивает формат данных InnoDB, за счет использования транзакций

```
CREATE TABLE CUSTOMER (A INT, B CHAR (20),  
INDEX (A)) TYPE = InnoDB;
```

Результатом представленной выше SQL-команды является создание таблицы и индекса в столбце A табличной области InnoDB.

В таблицах InnoDB все операции, которые выполняет пользователь, выполняются в виде отдельных транзакций, которые в последствии в MySQL выполняются автоматически.

Если же режим автоматической фиксации отключен, то предполагается использование постоянно открытой транзакции. Если же пользователь решает выполнить SQL -оператор COMMIT или ROLLBACK, завершающие текущую транзакцию, то вместо завершившейся транзакции тут же запускается новая транзакция. Два представленных оператора снимают все блокировки InnoDB, установленные во время выполнения текущей транзакции. Оператор COMMIT подтверждает все изменения, которые внес пользователь со времени запуска текущей транзакции, зафиксированы и стали видимы для других пользователей. Оператор ROLLBACK, в свою очередь, делает обратное оператору COMMIT, то есть он отменяет все изменения, внесенные со времени запуска текущей транзакцией.

Команда SHOW TABLE STATUS не предоставляет точных статистических данных по таблицам InnoDB. Исключением является размер физического пространства, которое было зарезервировано для таблицы.

```
SHOW TABLE STATUS IN T$ WHERE TNAME LIKE  
'%SUBSTRINGNAMETARGET%';
```

В дистрибутив исходного кода MySQL начиная с версии 3.23.34 стала поддерживать таблицы BDB. В университете Беркли была создана самая первая версия Berkeley DB. Транзакционный обработчик таблиц для среды MySQL обеспечивает BerkeleyDB. При использовании BerkeleyDB появляется большая возможность сохранения целостности таблиц при возникновении сбоев. Также они дают возможность осуществлять операции по управлению транзакциями, а именно COMMIT и ROLLBACK.

Пример использования **COMMIT**:

```
UPDATE PRODUCNTNAME SET CENA=300@ WHERE  
PRODUCT=400;  
SAVEPOINT POINT_SAVE;  
UPDATE PRODUCNTNAME SET CENA=300@;  
ROLLBACK TO SAVEPOINT POINT_SAVE;  
COMMIT;
```

Есть два файла с формированием этих типов таблиц при их создании. Файл создан с расширением .frm. Этот файл содержит структуру таблицы. Другой файл был создан с расширением .db. Храните данные и координаты.

Пример использования **ROLLBACK**:

```
USE NAMEDBASE;  
GO  
CREATE TABLE ZNACHENIYE_TABLE ([ZNACHENIYE]  
INTEGER);  
GO  
DECLARE @NAME_TRANSACTION# varchar2(120) =  
'NUMBER TRANSACTION ONE';  
BEGIN TRAN @NAME_TRANSACTION#  
    INSERT INTO ZNACHENIYE VALUES (1), (2);  
ROLLBACK TRAN 'NUMBER TRANSACTION ONE';  
INSERT INTO ZNACHENIYE_TABLE VALUES (3),(4);  
SELECT [ZNACHENIYE] FROM ZNACHENIYE_TABLE;  
DROP TABLE ZNACHENIYE_TABLE;
```

### Особенности таблиц BDB:

- Обеспечить откат транзакций. Файлы журналов сохраняются для таких таблиц. Эти файлы были сохранены при обнаружении отдельных баз данных.
- Удалите установленные контрольные точки и все файлы журналов при создании нового файла журнала. Кроме того, установка точек останова для таблиц BDB, следует в любое время выполнить команду FLUSH LOGS.
- Структура таблиц типа BDB – это дерево. Он снижает скорость чтения и увеличивает емкость, занимаемую на диске. Но к плюсам можно отнести быстрое действие при поиске конкретного значения.
- Все BDB должны иметь первичные ключи. Если пользователь не создал его. Среда создаст автоматически и будет использовать его скрытно. Длина такого ключа – 5 байт, но с увеличением числа увеличивается и емкость.
- Так как BDB ее поддерживает подсчет количества строк то SQL запрос на COUNT (\*) сработает медленно.
- Так как BDB древовидная структура, то запросы выполняются медленнее.
- Файлы между системами нельзя переносить обычный COPY-PAST

## 5. УПРАВЛЕНИЕ УЧЕТНЫМИ ЗАПИСЯМИ ПОЛЬЗОВАТЕЛЕЙ MYSQL

Методы добавления пользователей:

1. Команда GRANT.
2. Занесение напрямую в привелегии mysql.

Grant является более верным способом. Он прост и вызывает меньше ошибок.

INSERT INTO <имя пользователя> VALUES (<название сервера>, monty, <пароль> ('some\_pass'));

Команды GRANT и REVOKE позволяют создавать пользователей, раздавать и отнимать права доступа.

Пример

GRANT ALL PRIVILEGES ON \*.\* TO 'newuser'@'localhost'.

Представленные в этой команде звёздочки задают доступ пользователя к БД и таблице, соответственно. Команда в данном примере предоставляет пользователю возможность редактировать, читать и выполнять различные действия во всех БД и таблицах.

К перечню возможных значений параметра `priv_type` для команд GRANT и REVOKE относят:

- ALL.
- CREATE TEMPORARY TABLES.
- CREATE.
- ALTER.
- DELETE.
- DROP.
- SELECT
- INSERT.
- UPDATE.
- EXECUTE.
- INDEX.
- FILE.
- LOCK TABLES.
- PROCESS.
- REFERENCES.
- RELOAD.
- REPLICATION SLAVE.
- REPLICATION CLIENT.
- SHOW DATABASES. USAGE.
- SHUTDOWN.
- SUPER.

Команды, применяемые к таблицам:

- SELECT
- INSERT
- UPDATE

Пример:

```
UPDATE TABLENAME_EMPLOY SET employ = 56 WHERE depEmploy = 116;
```

- DELETE
- CREATE

Пример:

```
CREATE TABLE req (depl NUMBER (2),  
DnameREQ VARCHAR2(14), locNAMEreq  
VARCHAR2(13), create_date DATE DEFAULT  
SYSDATE);
```

- DROP
- GRANT
- INDEX
- ALTER.

Пример:

```
ALTER TABLE req MODIFY (employ VARCHAR2(50));
```

Команды, применяемые к столбцам:

- SELECT
- INSERT
- UPDATE.

Пример:

Создание пользователя monty:

```
GRANT_ALL_PRIVILEGES_ON_*. *_TO_monty@local  
host IDENTIFIED_BY_<Пароль>_WITH_GRANT_OPTION;  
GRANT_ALL_PRIVILEGES_ON_*. *_TO_monty@"%"_BY_<  
Пароль>_WITH_GRANT_OPTION;
```

Создание пользователя admin:

```
GRANT_RELOAD,_PROCESS_ON_*. *_TO_admin@loc  
alhost;
```

Admin – пользователь, который может подключиться с localhost без пароля. У него есть привилегии RELOAD и

PROCESS, которые могут запускать команды refresh, reload и flush-\*. Привилегии БД ему не назначаются, они выдаются со временем.

Тоже можно сделать следующим образом:

```
INSERT INTO user_VAL-UES('localhost','monty',  
PASSWORD(<Пароль>));_INSERT_IN-  
TO_user_VALUES('%','monty',PASSWORD(<Пароль>));_IN-  
SERT INTO user_SET_Host='localhost',User='admin',_Re-  
load_priv='Y',_Process_priv='Y';
```

Перед сохранением паролей в таблице user они шифруются. Поэтому INSERT должен применяться вместе с PASSWORD(<пароль>);

Глобальные привилегии задаются с помощью ON \*.\* , а привилегии БД с помощью ON db\_name.\*.

Чтобы определить права users с определенных компьютеров в MySQL, вы можете указать имя пользователя (user\_name) в форме user @ host. Если вам нужно указать пользовательскую строку, содержащую специальные символы (например, '-'), или строку хоста, содержащую специальные или групповые символы, вы должны разделить имя пользователя или удаленного компьютера на кавычки (например: 'test - user '@' test - hostname ').

В названии удаленного компьютера имеется возможность указания групповых символов. Так, например, user@"%.loc.gov" относится к user всех удаленных компьютеров домена loc.gov, а user@"144.155.166. %" относится к user всех удаленных компьютеров подсети 144.155.166.

В случае пренебрежения указанием оператора IDENTIFIED BY при создании нового пользователя, пользователь будет создан без пароля. Безопасность такого пользователя будет снижена.

Также имеется возможность задавать пароли используя команду SET PASSWORD.

```
SET PASSWORD FOR bob@"%.loc.gov" = PASSWORD  
("newpass");
```

Отмена привилегий осуществляется с помощью использования команды REVOKE. Синтаксис такой команды REVOKE аналогичен синтаксису команды GRANT.

Аннулирование привилегии CREATE и DROP пользователя tim в базе данных dba, представлены в примере ниже:

```
REVOKE_CREATE,_DROP_ON_dba.* FROM  
ex@target.com;
```

В случае, когда применяются привилегии для таблицы или столбца хоть и для одного пользователя, сервер будет проверять привилегии таблиц и столбцов для всех пользователей, что в несколько раз замедлит работу MySQL.

Все привилегии при запуске mysqld будут считаны в память. Привилегии базы данных, таблицы и столбца запускаются сразу, а привилегии пользователя – при следующем его подключении. Изменения в таблицах назначения привилегий, осуществляемые с использованием команд GRANT и REVOKE, обрабатываются сервером сразу. При изменении таблицы назначения привилегий вручную (с помощью использования команд INSERT, UPDATE и т.д.), возникает необходимость запуска оператора FLUSH PRIVILEGES или mysqladmin flush-privileges, для указания серверу в необходимости перезагрузки таблиц назначения привилегий.

## **6. ПРЕДОТВРАЩЕНИЕ СБОЕВ И ВОССТАНОВЛЕНИЕ**

Резервное копирование баз данных является главным вопросом администрирования СУБД. Поскольку все таблицы MySQL хранятся в виде файлов, то процесс выполнения резервного копирования не доставляет трудностей. Для согласованности резервной копии выбранных таблиц следует выполнить команды LOCK TABLES, а после чего FLUSH TABLES.

Команда LOCK TABLES позволяет блокировать указанные в ней таблицы для текущего потока. Команда

UNLOCK TABLES позволяет снять любые блокировки, имеющиеся в текущем потоке.

Команда FLUSH реализует очистку кэша, используемого MySQL.

Команда FLUSH TABLE предоставляет возможность записи на диск всех активных индексных страниц до того, как запустится процесс резервного копирования.

Для того, чтобы применять команду LOCK TABLES в среде MySQL версии 4.0.2, пользователю следует иметь глобальные привилегии LOCK TABLES и SELECT для необходимых таблиц. В версии 3.23 для такой процедуры потребуются привилегии SELECT, INSERT, DELETE и UPDATE для необходимых таблиц.

В случае, когда в потоке возникает блокировка операции READ для какой-либо таблицы, то чтение из такой таблицы предоставляется только в этом потоке. В случае блокировки операции WRITE для некоторой таблицы в потоке, только этот поток, в котором содержится блокировка, имеет возможность выполнять операции READ и WRITE по отношению к данной таблице. Все остальные потоки будут блокироваться.

Блокировка WRITE является приоритетнее блокировки READ, для максимально быстрой обработки изменений. Под этим подразумевается, что в случае блокировки операции READ в потоке и последующего запроса блокировки WRITE, остальные запросы на блокировку READ будут ожидать получения блокировки WRITE и ее снятия.

Если применяемые таблицы не будут поддерживать транзакции, то необходимо применять команду LOCK TABLES в случае ее обработки. В таком случае, можно быть уверенным, что никакой другой поток не запустился между операциями, например, SELECT и UPDATE. Применение операции LOCK TABLES представлено ниже:

- LOCK TABLES trans READ, customer WRITE;
- UNLOCK TABLES;

Если пренебрегать применением операции LOCK TABLES, то появляется вероятность вставки новой строки в

таблицу trans между выполнением операций SELECT и UPDATE каким-либо другим потоком управления.

Использование команды FLUSH TABLES WITH READ LOCK дает возможность блокировки всех таблиц во всех базах данных блокировкой READ.

В случае потребности выполнения резервного копирования на уровне SQL можно применить операцию SELECT INTO OUTFILE или BACKUP TABLE.

Оператор SELECT может быть представлен в форме SELECT ... INTO OUTFILE 'file\_name'. Такой вид команды записывает выбранные данные в файл, указанный в переменной file\_name. Этот файл будет создан на сервере и ранее такое же название не должен существовать. Для применения такого вида команды SELECT требуются привилегии FILE. Форма SELECT ... INTO OUTFILE изначально предназначена для выполнения очень быстрого дампа таблицы на серверном компьютере.

Создание резервной копии базы данных может осуществляться также с помощью программы mysqldump.

Представленная утилита предоставляет возможность получения дампа (“моментальный снимок”) содержимого одной базы данных или их совокупности для создания резервной копии или пересылки содержимого на другой SQL-сервер БД (не обязательно MySQL-сервер). В этом случае дмп будет включать в себя набор SQL-команд для создания и/или заполнения таблиц.

```
mysqldump [OPTIONS] database [tables]
```

Если использовать параметры --databases или --all-databases или не указывать названия таблиц, то получится дмп БД в целом (соответственно - всех баз данных).

В случае необходимости получения дампа всех баз данных, следует применить:

```
mysqldump --all-databases > all_databases.sql
```

### Синтаксис BACKUP TABLE

```
BACKUP TABLE tbl_name [, tbl_name...] TO  
'/path/to/backup/directory'
```

Такой подход позволяет копировать в каталог резервного копирования тот минимум табличных файлов, которого хватит для того, чтобы восстановить таблицы. На сегодняшний день применим только к таблицам MyISAM. Для таблиц MyISAM копирует файлы `‘.MYD’` (данных) и `‘.frm’` (определений).

При выполнении резервного копирования установится блокировка READ для всех необходимых таблиц на тот момент, пока копирование не будет закончено. Если резервное копирование необходимо выполнить в виде мгновенного образа нескольких таблиц, то первым делом следует запросить LOCK TABLES установки блокировки чтения для каждой таблицы в группе.

### Синтаксис RESTORE TABLE

```
RESTORE TABLE tbl_name [, tbl_name...] FROM  
'/path/to/backup/directory'
```

Такой подход предоставляет возможность восстановления таблиц или таблицы из резервной копии, которая была создана с применением команды BACKUP TABLE. В случае попытки восстановления таблиц, поверх уже существующих, будет выдана ошибка. Эти таблицы не перезаписываются.

## 7. КОМАНДЫ АДМИНИСТРИРОВАНИЯ БАЗ ДАННЫХ

### Синтаксис команды OPTIMIZE TABLE:

```
OPTIMIZE TABLE tbl_name [, tbl_name]
```

Представленная команда должна применяться после удаления большей части таблицы или в случае, когда в таблице было внесено большое количество изменений в строки переменной длины (таблицы, где имеются столбцы VARCHAR, BLOB или TEXT). Записи, будучи удаленными, поддерживаются с помощью связного списка, а следующие операции INSERT повторно используют позиции старых записей. Для перераспределения неиспользуемого пространство и дефраг-

ментации файла данных, есть возможность использования команды OPTIMIZE TABLE.

В настоящее время команда OPTIMIZE TABLE применяется только к таблицам MyISAM и BDB.

Синтаксис команды ANALYZE TABLE:

ANALYZE TABLE tbl\_name [, tbl\_name...]

Представленная команда сохраняет и анализирует распределение ключей для таблицы. В процессе проведения анализа таблица будет заблокирована для чтения. Такая функция применяется для таблиц MyISAM и BDB. Сохраненное распределение ключей выполняется, в случае принятия решения о порядке связывания таблицы.

Такая команда может заменять выполнение myisamchk -а для таблицы.

Синтаксис команды FLUSH:

FLUSH flush\_option [, flush\_option] ...

Команда FLUSH используется при очищении кэша, используемого MySQL. Чтобы запустить команду FLUSH пользователю следует иметь привилегии RELOAD.

Варианты параметра flush\_option представлены ниже в таблице.

| Название параметра | Значение                                                                                                         |
|--------------------|------------------------------------------------------------------------------------------------------------------|
| HOSTS              | Данный параметр необходим для того, чтобы почистить кэш на удаленных компьютерах.                                |
| LOGS               | Данный параметр нужен для того, чтобы открыть и закрыть файлы журналов с логами.                                 |
| PRIVILEGES         | Данный параметр нужен для того, чтобы все привилегии, находящиеся в среде были перезапущены через журнал в mysql |
| TABLES             | Данный параметр нужен для того, чтобы все активные таблицы в среде были закрыты в принудительном порядке         |

Синтаксис команды RESET:

RESET reset\_option [, reset\_option]

Команда RESET используется для очистки и применяется как усиленный вариант команды FLUSH.

Для запуска команды RESET, пользователю следует иметь привилегии RELOAD.

Варианты параметра reset\_option представлены ниже в таблице.

| Название параметра | Значение                                                                                                               |
|--------------------|------------------------------------------------------------------------------------------------------------------------|
| QUERY CACHE        | Данный параметр нужен для того, чтобы почистить все ответы сервера на различного рода запросы, которые хранятся в кэш. |
| MASTER             | Данный параметр нужен для того, чтобы все журналы были удалены, а индексы обновлены до начального значения.            |

Синтаксис команды KILL:

KILL thread\_id

Любое соединение с mysqld будет запущено в отдельном потоке. Для просмотра списка запущенных потоков используют команду SHOW PROCESSLIST. Для удаления потока применяют команду KILL thread\_id.

Для просмотра всех потоков необходимо обладать привилегией PROCESS. Для удаления потока необходимо обладать привилегией SUPER. В ином случае удалять и просматривать потоки можно только свои собственные.

Команда SHOW DATABASES предоставляет информацию по базам данных.

Команда SHOW STATUS предоставляет информацию о состоянии сервера, содержащая данные, представленные в таблице ниже.

| Название                                                                        | Значение                                                                                                                          |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Aborted_clients                                                                 | Данный параметр нужен для того, чтобы узнать количество отмененных соединений                                                     |
| Aborted_connects                                                                | Данный параметр нужен для того, чтобы узнать количество безуспешных попыток подключения к серверу                                 |
| Bytes_received                                                                  | Данный параметр нужен для того, чтобы увидеть, сколько байт было принято от каждого клиента за все время использования сессии     |
| Bytes_sent                                                                      | Данный параметр нужен для того, чтобы увидеть, сколько байт было отправлено каждому из клиентов за все время использования сессии |
| Connections                                                                     | Данный параметр нужен для того, чтобы узнать, сколько раз был произведен запрос на подключение к серверу                          |
| Delayed_writes                                                                  | Данный параметр нужен для того, чтобы узнать значение количества строк, вставленных после использования оператора INSERT DELAYED  |
| Flush_commands                                                                  | Данный параметр нужен для того, чтобы узнать, сколько раз была произведена очистка с использованием команды FLUSH                 |
| Handler_commit<br><a href="https://t.me/portalToIT">https://t.me/portalToIT</a> | Данный параметр нужен для того, чтобы узнать количество примененных внутренних команд COMMIT                                      |

| Окончание таблицы      |                                                                                                            |
|------------------------|------------------------------------------------------------------------------------------------------------|
| Handler_delete         | Данный параметр нужен для того, чтобы узнать количество удаленных строк в таблице                          |
| Questions              | Данный параметр нужен для того, чтобы узнать количество отправленных запросов на сервер                    |
| Slave_open_temp_tables | Данный параметр нужен для того, чтобы узнать количество открытых временных таблиц в текущий момент времени |
| Threads_cached         | Данный параметр нужен для того, чтобы узнать, сколько запущено потоков в кэше                              |
| Threads_connected      | Данный параметр нужен для того, чтобы узнать количество открытых потоков в текущий момент времени          |
| Threads_created        | Данный параметр нужен для того, чтобы показатель количества созданных потоков                              |
| Threads_running        | Данный параметр нужен для того, чтобы узнать, сколько потоков имеют активных характер                      |
| Uptime                 | Данный параметр нужен для того, чтобы узнать время работы сервера                                          |

Команда SHOW TABLE STATUS появилось с версии 3.23 и работает как SHOW STATUS, но предоставляет больше количества информации по каждой таблице. Пример выполнения данной команды:

## SHOW TABLE STATUS from kor

Команда `SHOW VARIABLES` позволяет отразить значения некоторых системных переменных MySQL. Получение такой же информации возможно с помощью команды `mysqladmin variables`.

При выполнении этой команды выводится различная информация, включая представленную в таблице ниже.

|                              |                                                                                       |
|------------------------------|---------------------------------------------------------------------------------------|
| <i>back_log</i>              | Показывает. Сколько команд может быть запущено в качестве запросов на одно соединение |
| <code>connect_timeout</code> | Показывает время ожидания пакета соединения                                           |
| <code>table_type</code>      | Показывает тип таблиц, который был присвоен по умолчанию                              |
| <code>version</code>         | Показывает версию сервера                                                             |
| <code>wait_timeout</code>    | Показывает, сколько ожидал сервер до его закрытия                                     |
| <code>tmpdir</code>          | Каталог временных файлов и таблиц                                                     |

Команда `SHOW LOGS` позволяет отразить информацию о состоянии существующих файлов журналов. На сегодняшний день выводится информация только по файлам журналов Berkeley DB (BDB).

Команда `SHOW [FULL] PROCESSLIST` предоставляет информацию о том, какие потоки запущены в текущий момент времени. Эта информация также может быть получена с применением команды `mysqladmin processlist`.

Команда `SHOW CREATE TABLE` показывает оператор `CREATE TABLE`, который будет создавать данную таблицу. Например:

`SHOW CREATE TABLE POLZOVATEL`

Create Table:

```
CREATE TABLE POLZOVATEL (  
identification int (11) default NULL auto_increment,  
firstNameUSER char (60) default NULL,  
PRIMARY KEY (id)  
) TYPE=MyISAM
```

## 8. ФАЙЛЫ ЖУРНАЛОВ MYSQL

Чтобы узнать, что происходит внутри mysqld можно воспользоваться журналами. Укажем несколько из них:

- Журнал ошибок.
- Журнал isam.
- Общий журнал запросов.
- Журнал обновлений log.
- Бинарный журнал обновлений.
- Журнал медленных запросов.

Если нас интересует информация, связанная с критическими ошибками, остановками и запусками стоит воспользоваться журналом ошибок.

Когда необходимо отладить код isam, пользуются журналом ISAM. Он хранит все изменения в таблицах ISAM.

Информация об установленных соединениях и выполненных запросах хранится в общем журнале запросов.

Все команды, меняющие данные, хранятся в журнале обновлений log. От этого типа журналов хотят в скором времени отказаться в пользу бинарного журнала обновлений.

Все команды, меняющие что-либо, хранятся в бинарном журнале обновлений. Основная область применения такого журнала – репликация. Отличается от журнала обновлений log тем, что хранит данные в более эффективном формате. Кроме этого, в нем фиксируется информация о времени выполнения каждого обновляющего запроса в базу данных. Их удобно использовать, когда выполняется репликация с подчиненного сервера (slave) через головной сервер (master). Для чтения такого лога используют утилиту mysqlbinlog.

Все запросы, на выполнение которых ушло больше времени чем указано в переменной `long_query_time`, попадают в журнал медленных запросов.

Все журналы хранятся в виде файлов в каталоге с данными `mysqld`.

Когда произошел какой-то сбой, для выяснения его причины удобно пользоваться журналом ошибок. Там логируются данные об остановках, запусках сервера и сообщения о критических ошибках. Кроме того, там можно найти ошибки запуска и завершения работы `mysqld`.

Ошибки в `mysqld` пишутся на диск в виде файла с расширением `.err`. Для операционных систем семейства Windows путь к файлам ошибок будет таким: ``\mysql\data\mysql.err'`.

В этом файле можно прочесть данные о запуске и завершении работы `mysqld`, а так же обо всех критических ошибках, возникших при работе. В случае возникновения ситуации, когда происходит аварийное завершение работы и компонент `safe_mysqld` будет вынужден перезагрузить `mysqld`, то информация об этом событии запишется в файл.

Этот журнал выполняет еще одну функцию – сохраняет предупреждение в том случаи, когда `mysqld` находит таблицу, с которой нужно выполнить проверку или исправление.

Файлы настроек журнала можно найти в файле `my.cnf`. Лог может быть найден по следующему пути: `log_error = /var/log/mysql/mysql.err`.

С помощью флага `log_warnings` можно указать, нужно ли выписывать в лог предупреждения. Если нужно, то необходимо указать `log_warnings = 1`, если нет - `log_warnings = 0`

При запуске с параметром `--log-slow-queries[=file_name]` `mysqld` создается журнал медленных запросов, в который заносят те SQL запросы, выполнение которых занимает по времени больше, чем указано в значении параметра `long_query_time`.

Его используют для того, чтобы выявить слишком емкие запросы. Обнаружение таких запросов – первый шаг к созданию оптимизированной системы. Используя этот лог можно выявить те запросы, которые нужно оптимизировать в

первую очередь. Если приложение работает медленно стоит заглянуть в него.

Настраивается этот журнал с помощью файла `my.cnf`.

Лог может быть найден в следующей директории:  
`log_slow_queries = /var/log/mysql/mysql_slow.log`

Для того, чтобы определить порог, при превышении которого запрос считается медленным используется переменная `long_query_time`. Минимальное значение, которое можно указать – 1 секунда. Значение по-умолчанию – 10 секунд.

В общем журнале запросов можно найти информацию об установленных соединениях и завершенных запросах. Если запустить приложение с ключом `--log[=file]`, то в нем будет указываться все, что происходит с `mysqld`. По-умолчанию файлу журнала дается имя `hostname.log`. С помощью этого журнала можно анализировать и локализовать ошибки в клиентских приложениях.

В лог пишется вся информация о подключениях и отключениях клиентов, кроме того, указываются все SQL запросы, которые были получены. Этот лог так же называют временным по причине того, что он удаляется в автоматическом режиме сразу по окончании выполнения команд. Лог пишется в той же очередности, которой запросы и поступают. Он содержит абсолютно все запросы к базе данных независимо от того они пришли (от приложения, от пользователя, от администратора и др.). С помощью этого лога так же можно отлавливать те запросы, которые вызвали сбой (например, когда генерируется динамический SQL). Для настройки этого типа журналов используется файл `my.cnf`. Лог расположен по следующему пути: `log = /var/log/mysql/mysql.log`.

## 9. РЕПЛИКАЦИЯ В MYSQL

Репликация – процесс, представляющий собой тиражирование изменений данных с главного сервера БД (его называют мастер-сервер) на одном или нескольких зависимых серверах (реплики).

Когда на мастере происходят какие-то изменения, то эти изменения применяются и для реплик. Обратное действие не верно – при внесении изменений на реплике на мастер сервере изменения будут отсутствовать. Поэтому, запросы, связанные с изменением данных, такие как UPDATE, DELETE, INSERT выполняются только на мастере. Запросы на чтение, то есть запросы SELECT могут выполнять как реплики, так и сам мастер сервер. Репликация одной из реплик никак не влияет на другие реплики и в незначительной степени влияет на мастера.

Механизм репликации реализуется путем ведения бинарных логов на мастере. В них записываются все запросы, которые могут привести или приводят к изменениям в базе данных. Бинарные логи можно прочитать только, используя специальную утилиту – mysqlbinlog из-за того, что они пишутся в бинарном формате. Специальные логи, называемые биндлоги, передаются на реплики и сохраненные запросы выполняются последовательно, начиная с определенной позиции. В процессе репликации содержимое базы данных дублируется на несколько серверов. Этот механизм обладает рядом ощутимых преимуществ:

- **Производительность и масштабируемость.** Разбиение нагрузки между несколькими серверами очень сильно увеличивает общую производительность системы. Создание реплик особенно полезно, когда большинство запросов – запросы на выборку.

- **Отказоустойчивость.** Если реплика перестанет отвечать, то ее нагрузка будет распределена между другими репликами и мастером. Если мастер перестанет отвечать, то его задачи может начать выполнять одна из реплик. В таком случае, после того как мастер будет восстановлен, он станет играть роль реплики.

- **Резервирование данных.** Для создания резервной копии данных можно приостановить работу реплики. Приостановить мастер нельзя!

– **Отложенные вычисления.** Для выполнения тяжелых запросов можно отвести специальную реплику, чтобы не замедлять работу системы в целом.

MySQL, начиная с версии 3.23.15 добавлен односторонний внутренний механизм репликации. Один сервер действует как головной, а другие - как подчиненные.

При реплицировании базы данных все обновляющие и добавляющие данные запросы должны выполняться через мастер-сервер!

К неоспоримым преимуществам репликации так же можно отнести и то, что каждая реплика является «живой» резервной копией системы и можно выполнять резервное копирование на реплике, а не на мастер-сервере.

Механизм репликации в MySQL основан на том, что каждое изменение в базе данных, произошедшее на мастер-сервере заносится в протокол в двоичном виде. Сервер-реплика читает данный протокол, вносит изменения в свою копию данных, таким образом поддерживается актуальное состояние.

Создания подчиненного сервера нужно сначала скопировать с мастер-сервера все данные, существовавшие на нем на тот момент. Если реплика будет запущена с данными, которые не будут соответствовать тем, что хранятся на мастере, то на подчиненном сервере может произойти сбой.

Для упрощения этой процедуры, начиная с версии 4.0.0 для записи данных на подчиненный сервер, удобно использовать одну команду: `LOAD DATA FROM MASTER`. Эта команда создает образ мастер-сервер и копирует его на реплику.

Если сервер-реплика правильно сконфигурирован, то он должен легко запуститься и соединиться с мастер-сервером. После этого должно произойти обновление. В случае, если оборвется связь между серверами или мастер-сервер не будет отвечать, то подчиненный сервер будет пытаться восстановить соединение каждый раз. Когда истекает время от предыдущей попытки, указанное в переменной `master-connect-retry` (в секундах) (в `my.ini` (windows) или `my.cnf`(unix)) до тех

пор, пока не установится соединение и не произойдет синхронизация с получением обновлений.

Каждая реплика отслеживает события с момента разрыва соединения. Мастер-сервер не обладает никакой информацией о том, сколько ему подчинено реплик, какие из них обновлены последними данными в отдельный отрезок времени.

Чтобы выполнить репликацию необходимо выполнить такой набор команд.

Настройки мастера:

**1 шаг.** Требуется назначить уникальный идентификатор для каждого сервера, прописать пути к бинарным логам и название базы данных для репликации в секции [mysqld] файла my.cnf на мастер-сервере.

```
[mysqld]
```

```
log-bin = /log/bin.log
```

```
server-id=1
```

```
replicate-do-db = replica
```

Обязательным условием является то, чтобы ID мастер-сервера отличался от ID серверов-реplik. Часто полагают что server-id играет роль IP-адреса. Он уникально идентифицирует сервер среди участников репликации.

log-bin=filename - Указывает местоположение двоичного журнала обновлений, в котором будут вестись записи.

Возможны еще параметры:

max\_binlog\_size= 500M - Максимальный размер, минимум 4096 байт, по умолчанию 1073741824 байт (1 гигабайт):

expire\_logs\_days = 3 - Задаёт период хранения (Сколько дней хранится лог).

**2 шаг.** Создать пользователя и дать ему полномочия на репликацию:

```
GRANT replication slave ON "testdb".* TO "replication"@"192.168.1.102" IDENTIFIED BY "password".
```

**3 шаг.** Создаём копию реплицируемой базы:

```
$mysqldump -uroot -p replica > replica.dump
```

4 шаг. И переносим эту копию на хост № 2.

На 2-м хосте создаём базу replica и наполняем её из копии:

- `$mysqladmin -uroot -p create replica`
- `$mysql -uroot -p replica < replica.dump`

В файле конфигурации MySQL на подчиненном сервере (/etc/my.cnf): добавляем параметры:

- `master-host=mysql2`
- `server-id = 2`
- `master-user=slave_user`
- `master-password=slave`
- `master-connect-retry=60`
- `replicate-do-db=replica`
- `relay-log=/log/slave-relay-bin`
- `relay-log-index=/log/slave-relay-bin.index`

`master-host=host` - имя хоста мастер-сервер или IP-адрес для репликации. Заполнение этого поля является обязательным условием.

`master-user=username` – указывает пользователя, под которым будет выполняться вход при синхронизации серверов. Для этого пользователя на мастера-сервер должны быть выданы специальные права FILE. Если не указать это поле, то будет выбрано значение по-умолчанию – `test`.

`master-password=password` – указывает пароль пользователя, указанного в строчке `master-user`. По умолчанию пароль пустой.

`master-connect-retry=seconds` - время ожидания в секундах для потока подчиненного главного перед повторением попытки. По умолчанию - 60. Пример: `master-connect-retry=60`

`replicate-do-db=database_name` – сообщает подчиненному серверу, что реплицироваться должна только указанная база данных. Чтобы указать более одной базы данных, директиву следует использовать несколько раз, по одному разу для каждой базы данных.

master-port=portnumber - порт, который слушает головной сервер. Это должно быть значение 3306, если оно не было изменено при помощи опций `configure`.

relay-log и relay-log-index – задают пути к log файлам.

**5 шаг.** Перезапустить подчиненные серверы.

## **10. ЛАБОРАТОРНЫЕ РАБОТЫ**

### **ЛАБОРАТОРНАЯ РАБОТА № 1**

#### **Создание баз данных и таблиц в среде MYSQL.**

#### **Информационное наполнение базы данных**

##### **Цель работы**

Целью выполнения лабораторной работы является ознакомление с приложениями, включенными в состав СУБД MySQL. Между прочим, в ходе работы будут получены навыки управления данными учета записей пользователей, а также, определения привилегий этих пользователей. Произойдет навыки работы с ними. Ознакомитесь с возможностями СУБД MySQL и создадите с его помощью базу данных, которая будет содержать набор таблиц, которые необходимо заполнить данными для последующей работы.

##### **Задание на лабораторную работу**

###### **Задание 1**

1.1. Для начала работы необходимо запустить сервер MySQL. Пользуясь консольным представлением, создайте пользователя, выдайте ему необходимые права.

1.2. Для того, чтобы оценить структуру и состав таблицы воспользуйтесь утилитой `Mysqlshow`. В отчет включите схему таблицы.

1.3. Чтобы сделать полный дамп базы данных MySQL воспользуйтесь утилитой `Mysqldump`. Получите полный дамп, дамп отдельных таблиц и данных.

## Задание 2

2.1. Познание базового уровня работы с приложением-клиентом под названием MySQL.

2.2. Прочитать про основные команды языка SQL, которые дают в базе данных, занимается удалением, вставкой, модификацией записей таблицы.

| <b>Функция</b>                            | <b>Описание</b>                                 |
|-------------------------------------------|-------------------------------------------------|
| <u>CREATE DATABASE</u><br>ИМЯ_БАЗЫ_ДАННЫХ | создание базы данных                            |
| <u>USE DATABASE</u><br>ИМЯ_БАЗЫ_ДАННЫХ    | выбор существующей базы данных                  |
| <u>CLOSE DATABASE</u><br>ИМЯ_БАЗЫ_ДАННЫХ  | заккрытие файлов текущей базы данных            |
| <u>DROP DATABASE</u><br>ИМЯ_БАЗЫ_ДАННЫХ   | удаление базы данных                            |
| <u>CREATE TABLE</u><br>ИМЯ_БАЗЫ_ДАННЫХ    | создание таблицы базы данных                    |
| <u>ALTER TABLE</u><br>ИМЯ_БАЗЫ_ДАННЫХ     | модификация структуры базы данных               |
| <u>DROP TABLE</u><br>ИМЯ_БАЗЫ_ДАННЫХ      | удаление таблицы базы данных                    |
| <u>INSERT</u>                             | добавление одной или нескольких строк в таблицу |
| <u>DELETE</u>                             | удаление одной или нескольких строк из таблицы  |
| <u>UPDATE</u>                             | модификация одной или нескольких строк таблицы  |
| <u>LOAD DATA INFILE</u>                   | загрузка данных в таблицы из файла              |

2.3. Создать базу данных.

Для того, чтобы создать базу данных в MySQL можно воспользоваться утилитой mysqladmin. По-умолчанию даются

две базы: `mysql`, в которой лежат настройки, предназначенные строго для администратора и база данных `test`. Она по умолчанию пустая и каждый имеет к ней доступ. Ниже показан простейший сценарий, направленный на создание новой базы данных.

```
Mysql/bin>mysqladmin -u root -p create data_name
```

```
Enter password: *****
```

```
Database "data_name" created.
```

```
mysqlbin>
```

Тут `data_name` – имя создаваемой базы данных. Чтобы проверить создана ли база данных или увидеть список уже существующих баз данных можно воспользоваться командой **Show databases**. Для этого так же можно воспользоваться утилитой **mysqlshow**.

По умолчанию, пользователь с логином **root** имеет все привелегии и возможности. Чтобы перейти в созданную базу данных пользуются командой **mysql**. Use database

---

```
Mysql/bin>mysql -u root -p data1
```

```
Enter password: *****
```

```
Welcome to MySQL monitor.
```

---

Чтобы перейти из одной базы данных в другую нужно ввести команду:

---

```
mysql>use data1
```

```
Database changed.
```

---

Из клиентского приложения так же можно создать базу данных, введя команду:

### ***CREATE DATABASE ИМЯ\_БАЗЫ\_ДАННЫХ***

Тут ***ИМЯ\_БАЗЫ\_ДАННЫХ*** имя новой базы данных. После команды БД готова к работе, в ней можно создавать информацию и вводить таблицы. Эти операции можно выполнить с помощью специального программного обеспечения, такого как, например, MySQL-Front. Чтобы запустить его выберите ПУСК/ПРОГРАММЫ.

Необходимо указать:

- имя пользователя бд;
- хост расположения бд;
- пароль пользователя бд;
- порт, на котором располагается бд;
- имя бд (при необходимости) к которой будет выполнено подключение.

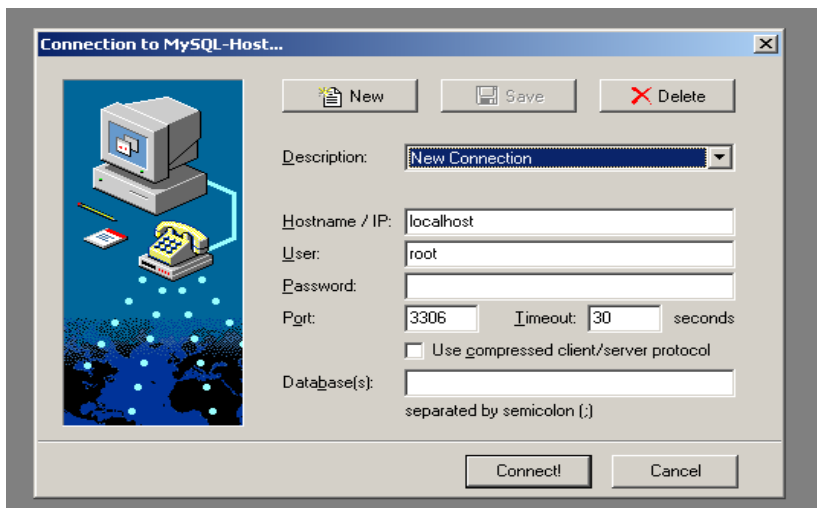
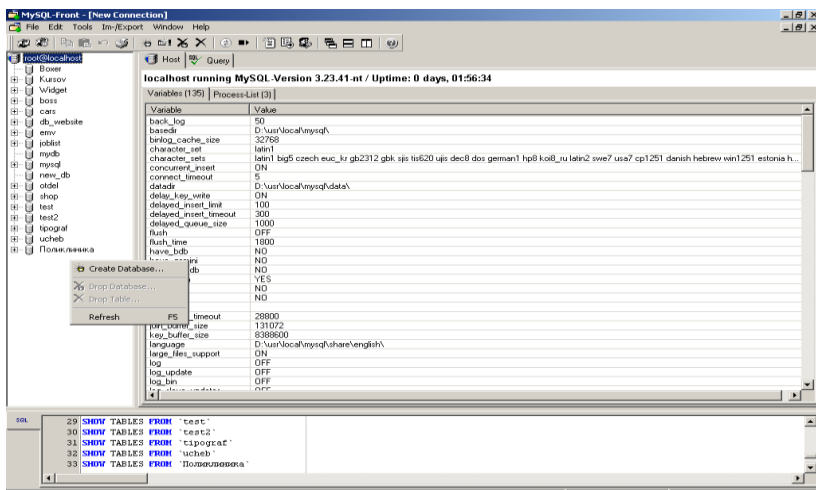


Рис. 1. Запуск MySQL-front



Указав активную базу данных с помощью программы можно изменять структуру базы данных, можно вводить данные, указывать ключевые поля. Кроме того, с помощью модального отображения можно переносить инструкции напрямую с применением языка SQL. Пример показан на рисунке:

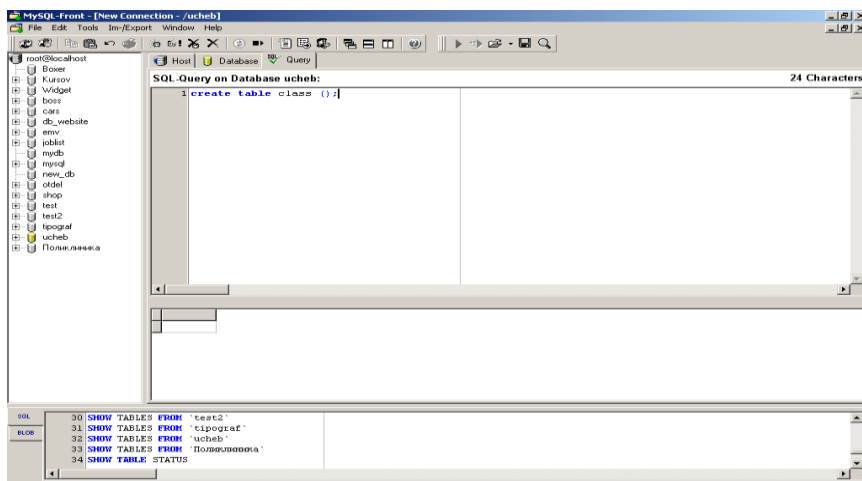


Рис. 3. Использование синтаксиса SQL

2.4. Пользуясь командой языка SQL CREATE TABLE создайте, четыре таблицы в базе данных. Синтаксис приведен в приложении. Например, для таблицы EMPLOYEE:

```
CREATE TABLE EMPLOYEE (  
  EMPLOYEE_ID NUMBER (6,0) NOTNULL,  
  FIRST_NAME VARCHAR (20) NOTNULL  
  LAST_NAME VARCHAR (20),  
  PHONE_NUMBER VARCHAR (20))
```

С помощью символов `/* */` в SQL команде можно выделить участки текста, которые не будут учитываться при выполнении запроса – комментарии.

Когда будете создавать таблицы, придерживайтесь той структуры, которая бы отражала структуру из таблиц ниже (таблицы S, P, J, SPJ). Кроме того, на них должны быть наложены следующие ограничения:

- поля номер\_поставщика, номер\_детали, номер\_изделия в каждой таблице должен иметь цифробуквенное значение, состоящее из 6 символов (**varchar (6)**);
- поля рейтинг, вес и количество сделать целочисленными (**integer**);
- поля фамилия, город (поставщика, детали или изделия), название (детали или изделия) могут состоять из 20 букв и цифр (**varchar (20)**);
- все поля неиндексируемые;
- все поля, кроме первичных и внешних ключей могут иметь значения null и дублироваться.

После того как база данных создана и можно пользоваться, однако в таблице нет данных. Введем туда данные. Этот процесс может происходить несколькими способами:

а) вручную, с применением команды **insert into**;

Пример ввода данных вручную (команда INSERT):

```
mysql> INSERT INTO EMPLOYEES
```

```
VALUES ('1', 'Иван', 'Иванов', '87654321987', 30000,  
45)
```

или

```
mysql> insert into J values ('J1', 'Жесткий  
диск', 'Париж');
```

//примечание: если вы хотите вставить данные во все поля таблицы, то сами поля перечислять не обязательно.

Вставим id и имя в таблицу:

```
INSERT INTO table_name (id, name) VALUES  
('id_value', 'name_value');
```

Итак, общая структура запроса, направленного на введение данных в таблицу данных, имеет следующий вид:

```
insert into имя_таблицы [(поле [, поле] ...)] values  
(константа [, константа] ...)
```

б) мы можем получить данные из текстового файла. Этот способ является более быстрым и менее трудозатратным. Он особенно актуален, когда требуется внести большое число данных.

Синтаксис команды LOAD DATA INFILE.

```
DATA [LOW_PRIORITY] [LOCAL] INFILE  
'file_name.txt' [REPLACE | IGNORE]  
INTO TABLE tbl_name  
[FIELDS  
[TERMINATED BY 't']  
[OPTIONALLY] ENCLOSED BY " ]  
[ESCAPED BY " ]]  
[LINES TERMINATED BY 'n']  
[IGNORE number LINES]  
[(col_name,...)]
```

### **Пример**

```
LOAD DATA LOCAL INFILE '/MyDocs/categories.txt'  
REPLACE  
INTO TABLE category FIELDS TERMINATED BY ';' OP-
```

TIONALLY ENCLOSED  
BY \"\" LINES TERMINATED BY \"\n\"

В вышеуказанном примере данные находятся в файле *categories.txt*, расположенном на компьютере в директории *C:\MyDocs*.

Когда применяется система, основанная на UNIX, то слово

## REPLACE

В SQL запросе означает, что необходимо замещать записи с совпадающими значениями ключей.

## INTO TABLE

указывает на таблицу, куда будут импортированы данные.

FIELDS TERMINATED BY ';'

указывает разделители полей, порядок полей должен быть таким же, как и в таблице назначения,

OPTIONALLY ENCLOSED BY \"\"

указывает, что поля VARCHAR взяты в двойные кавычки, и

LINES TERMINATED BY \"\r\"

в) можно использовать специальную утилиту *mysqlimport* для получения данных из текстового файла.

Кроме консольного представления в MySQL есть и графическая программа MySQL-Front. Она также может выполнить эту операцию.

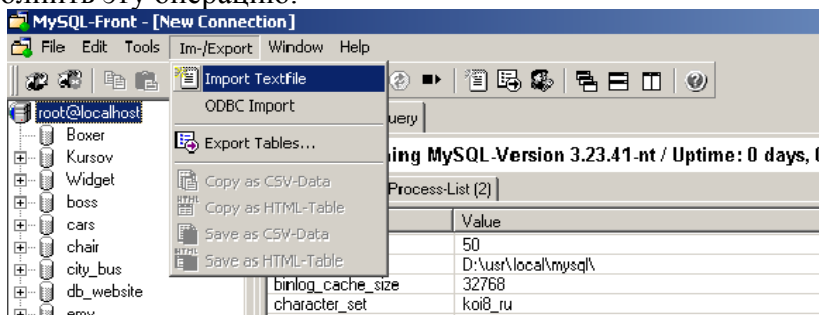


Рис. 4. Использование программы MySQL-front для заполнения таблиц данными из файла

Таблица поставщиков (S)

| Номер поставщика | Фамилия | Рейтинг | Город   |
|------------------|---------|---------|---------|
| S1               | Смит    | 20      | Белград |
| S2               | Джонс   | 10      | Париж   |
| S3               | Блейк   | 30      | Париж   |
| S4               | Кларк   | 20      | Белград |
| S5               | Адамс   | 30      | Сумы    |

Таблица деталей (P)

| Номер детали | Название | Цвет    | Вес | Город   |
|--------------|----------|---------|-----|---------|
| P1           | Шайба    | Красный | 12  | Белград |
| P2           | Болт     | Зеленый | 17  | Париж   |
| P3           | Вороток  | Голубой | 17  | Москва  |
| P4           | Вороток  | Красный | 14  | Белград |
| P5           | Кулачок  | Голубой | 12  | Париж   |
| P6           | Блюм     | Красный | 19  | Белград |

Таблица изделий (J)

| Номер изделия | Название     | Город  |
|---------------|--------------|--------|
| J1            | Жесткий диск | Париж  |
| J2            | Перфоратор   | Москва |

Окончание таблицы

|    |             |         |
|----|-------------|---------|
| J3 | Считыватель | Сумы    |
| J4 | Принтер     | Сумы    |
| J5 | Флоппи-диск | Белград |
| J6 | Терминал    | Осло    |
| J7 | Лента       | Белград |

Таблица поставок (SPJ)

| Номер поставщика | Номер детали | Номер изделия | Количество |
|------------------|--------------|---------------|------------|
| S1               | P1           | J1            | 200        |
| S1               | P1           | J4            | 700        |
| S2               | P3           | J1            | 400        |
| S2               | P3           | J2            | 200        |
| S2               | P3           | J3            | 200        |
| S2               | P3           | J4            | 500        |
| S2               | P3           | J5            | 600        |
| S2               | P3           | J6            | 400        |
| S2               | P3           | J7            | 800        |
| S2               | P5           | J2            | 100        |
| S3               | P3           | J1            | 200        |
| S3               | P4           | J2            | 500        |
| S4               | P6           | J3            | 300        |
| S4               | P6           | J7            | 300        |

Окончание таблицы

|    |    |    |     |
|----|----|----|-----|
| S5 | P2 | J2 | 200 |
| S5 | P2 | J4 | 100 |
| S5 | P5 | J5 | 500 |
| S5 | P5 | J7 | 100 |
| S5 | P6 | J2 | 200 |
| S5 | P1 | J4 | 100 |
| S5 | P3 | J4 | 200 |
| S5 | P4 | J4 | 800 |
| S5 | P5 | J4 | 400 |
| S5 | P6 | J4 | 500 |

Выполнить все задания, проверить правильность выполнения. Если возникли ошибки, исправить. Для увеличения скорости введения данных в базу рекомендуется воспользоваться командой `LOAD DATA` (синтаксис в приложении). Чтобы файл имел правильный формат данных их сначала нужно скопировать из электронных таблиц Excel, в текстовые данные.

2.5. Модифицируйте таблицу SPJ, таким образом, чтобы в нем добавилось поле, где будет храниться дата поставки. Проверить, что поле создано. При возникновении ошибок, исправить. Пользоваться командой `Alter Table`.

2.6. Удалить все таблицы вместе с данными. Предварительно необходимо сделать резервную копию базы данных и ее содержимого. Используйте средства работы СУБД. Проанализировать результаты выполнения.

2.7. Напишите и выполните запросы для модификации таблиц таким образом, чтобы они отвечали концептуальной модели, показанной на рис. 5. Проверить корректность вы-

полнения задания. В случае возникновения ошибок – исправить. Проанализировать результаты.

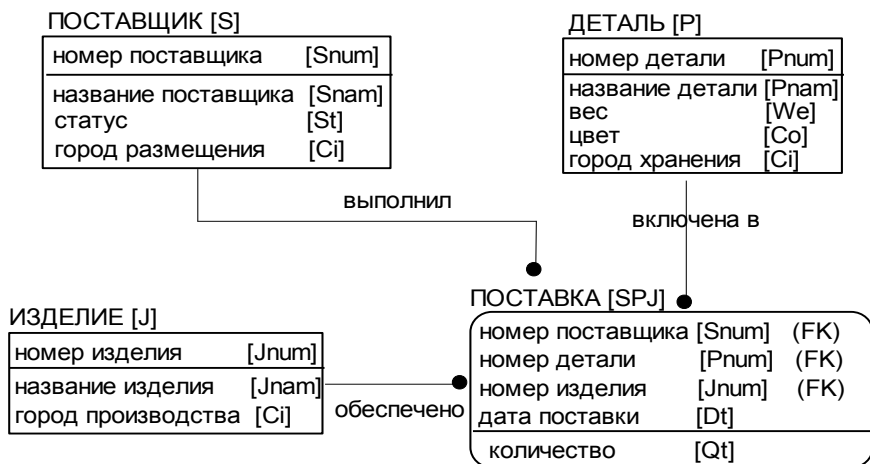


Рис. 5. Концептуальная модель учебной базы данных

Чтобы проверить, какие данные хранятся в таблице на текущий момент времени, выполните запрос:

***select \* from имя\_таблицы***

В случае обнаружения ошибок исправить их, изменив данные либо удалив их из таблицы.

### Контрольные вопросы

1. Каким способом возможен запуск серверной части СУБД.
2. Что такое привилегия. Каково её предназначение.
3. Какие основные утилиты входят в состав СУБД, какие функции они выполняют.
4. В каких режимах возможно создание базы данных?
5. Какие типы данных допустимы при создании таблицы?

6. Как выполнить создание таблицы средствами СУБД?
7. Как выполнить создание таблицы средствами языка SQL?
8. Как разделяются операторы SQL в случае нескольких операторов в запросе?
9. Каким образом выполнить просмотр таблицы?
10. Как получить информацию о структуре таблицы в рамках СУБД MySQL?

## **ЛАБОРАТОРНАЯ РАБОТА № 2**

### **Работа с командами резервного копирования, администрирования и восстановления БД**

#### **Цель работы**

Целью лабораторной работы является ознакомление с основами администрирования MySQL при помощи командной строки.

#### **Методические указания к выполнению работы**

**MySQL** является популярной системой управления базами данных. Она применяется для огромного спектра различных нужд. Например, эта СУБД принята как стандарт в области интернет хостинга. Так же популярной является СУБД **phpMyAdmin**. Эта СУБД является хорошим, удобным продуктом, однако требует специальных навыков работы с ней. Зачастую, эти навыки перекрывают навыки работы с самой СУБД. Эта лабораторная работа знакомит студентов с основными задачами процесса администрирования MySQL с помощью командной строки.

Лабораторная работа преследует цель научить студента использовать именно консольное представление, а не различного вида панели управления, насколько бы удобны они не были. Это не означает что они плохие, важно научить работать с примитивными представлениями.

Основное отличие консольного представления от командной строки состоит в том, что человек, который умеет работать с использованием командной строки, использует панель управления не, потому что он не умеет работать с консолью, а лишь делает проще ежедневный труд. Наоборот, тот человек, который умеет работать только с панелью управления, при ее отсутствии не сможет сделать ничего, так как не понимает, что хочет от него консоль.

На деле же, командная строка, с помощью которой можно управлять СУБД является несложным для освоения компонентом. К тому же, часть задач, связанных с администрированием проще выполнить именно через консоль. Под администрированием будем понимать администрирование сервера системы управления базами данных, а не баз данных. Для работ с базой данных, наоборот, рекомендуется использовать подходящие инструменты, но это не исключает администрирование и через консоль.

### **Задание на лабораторную работу**

1. Настроить правление базами данных и пользователями в MySQL
2. Выполнить проверку, оптимизацию, исправление ошибок баз данных MySQL с помощью команд.
3. Реализовать процедуру восстановления данных и паролей.

### **Контрольные вопросы**

1. Приведите перечень команд и их назначение для выполнения функций копирования, восстановления и оптимизации базы данных в MySQL.

## ЛАБОРАТОРНАЯ РАБОТА № 3

### Работа с командами получения информации по базам данных, таблицам, столбцам и индексам

#### Цель работы

С помощью базы данных, созданной на предыдущих лабораторных работах написать и выполнить набор запросов, направленных на выборку информации, а так же выполнить модификацию данных в самой базе.

#### Методические указания к выполнению работы

1. Ниже приведены основные команды языка SQL. С их помощью создаются запросы для добавления, модификации, удаления и выборки информации:

**select** – с его помощью можно получить информацию;

**insert** – с его помощью информация добавляется в базу;

**delete** – с его помощью информация удаляется из базы данных;

**update** – с его помощью выполняется модификация данных в базе данных;

**union** – с его помощью можно объединить несколько выборок в одну.

2. Оператор **select** применяется в комбинации с набором ключевых фраз для уточнения выдаваемой информации:

**select** – описывает какие столбцы с данными будут введены в результирующий набор. Оператор является обязательным;

**from** – описывает одну или несколько таблиц, откуда будут выбираться данные. Оператор является обязательным;

**where** – описывает фильтры, которые могут накладываться на выбираемые данные;

**group by** – оператор, описывающий логику при группировке выбранных данных. Группой принято называть несколько строк, имеющих одинаковые значения в одном или нескольких столбцах;

**having** – оператор, описывающий фильтрацию, выполняемую после группировки. Оператор используется только совместно с group by;

**order by** – оператор, описывающий логику сортировки выборки;

**into outfile** – оператор используемый, когда необходимо вывести результат запроса в файл.

### **Задание на работу**

1. Написать и выполнить 3 запроса, применяя различные функции работы с различными типами данных (дата, строковые значения). Хотя бы один из них должен содержать группировку.

2. Написать и выполнить 4 запроса, направленные на выборку информации из базы данных с применением агрегатных функций.

3. Написать и выполнить 2 запроса с применением операторов, направленных на модификацию информации (вставка, удаление, обновление). Варианты для выполнения заданий представлены ниже. Если ваш вариант содержит создание таблиц, то предполагается что будет создана постоянная таблица.

## **Варианты заданий на составление запросов по выборке информации из таблиц базы данных**

### **Вариант 1**

1. Выполняется поставка изделия А, собранного из деталей. Выведите номер изделия А, номер детали, а также общее число поставляемых деталей этого вида.

2. Выведите следующие поля: "номер поставщика", "номер детали" и "номер изделия", причем такие что, деталь, изделие и поставщик не являются соразмещенными в одном городе.

3. Покажите все номера изделий, которые состоят только из деталей одного конкретного поставщика S1. Детали от поставщика S1 должно быть достаточно чтобы укомплектовать изделие. В базовом наборе можно найти все изделия.

Комплектующие изделия могут быть найдены в базовом наборе данных таблицы поставка.

4. Вывести все фамилии поставщиков и их номера, которые поставляют детали для любого изделия, в составе которого есть деталь P1. В таком объеме, который превышает средний объем поставки детали P1 для этого изделия.

### **Вариант 2**

1. Посчитать и вывести общее число деталей P1 от поставщика S1.

2. Вывести названия городов-источников и городов-импортеров, такие, что какой-то поставщик покупает детали в одном городе и продает их в другом.

3. Выведите номера изделий, состоящих только их деталей поставщика S1.

4. Выведите номера всех деталей, которые изготавливаются в Белграде и применяются в изделиях, изготавливаемых так же в Белграде.

### **Вариант 3**

1. Выберите фамилии поставщиков и их номера, которые выполняют поставку одной и той же детали для всех указанных изделий. Изделия, для которых нужно выполнить это укажет преподаватель.

2. Посчитайте, и укажите, сколько изделий есть в базе, детали для которой поставляет поставщик S1.

3. Выведите номера тех изделий, детали для которых поставляет каждый из поставщиков, и он же поставляет хотя бы одну красную деталь. Вам необходимо узнать номера изделий, детали которых komponуются от всех поставщиков. К тому же хотя бы одна деталь этого поставщика имеет красный цвет.

4. Выведите “номер поставщика”, “номер детали” и “номер изделия”, при том, что в каждой тройке поставщик деталь и изделие являются размещенными в одном городе парно.

### **Вариант 4**

1. Выберите фамилии и номера поставщиков, которые поставляют не менее одной детали, которую поставляют не менее одного поставщика, при условии, что в его ассортименте есть хоть одна деталь красного цвета. Таким образом, необходимо выдать информацию о поставщиках, имеющих возможность поставки таких деталей, которые могут поставить поставщики, поставляющие хотя бы одну красную деталь.

2. Вывести полный список деталей, необходимых для каких-либо изделий, которые производятся в Белграде.

3. Укажите номера тех деталей, которые поставляет поставщик родом из Белграда.

4. Укажите номера деталей для любого изделия, которые сделаны в Белграде.

### **Вариант 5**

1. Укажите номера тех изделий, для которых идут поставки минимум одним поставщиком, родом не из того же города, что и само изделие

2. Выведите поставки деталей, размер которых более 300, но менее 750 включительно.

3. Укажите номера изделий, при производстве которых необходима хотя бы одна деталь от поставщика S1. Вам нужно получить те изделия, которые в производстве используют хотя бы одна деталь от поставщика S1.

4. Укажите название деталей и номера, которые необходимы для хотя бы одного изделия из Белграда.

### **Варианты заданий на составление запросов по модификации информации из таблиц базы данных**

#### **Вариант 1**

1. Всем поставщикам, рейтинг которых меньше рейтинга поставщика S4 нужно увеличить рейтинг в 10 раз.

2. Создайте новую таблицу и поместите туда те номера изделий, которые либо находятся в Белграде, либо хотя бы

одна деталь для него поставляется поставщиком родом из Белграда.

### **Вариант 2**

1. Исключить из базы данных все изделия, детали для которых не поставляются.

2. Создайте новую таблицу и поместите туда номера поставщиков и пары номеров деталей, которые поставляет этот поставщик. Учесть, что комбинации должны быть без повторений.

### **Вариант 3**

1. Для всех поставщиков, которые поставляют хотя бы одну красную деталь увеличить размер поставки на 10 процентов.

2. Создайте новую таблицу и заполните ее парами "цвет детали-города, где хранится деталь". Исключите возможность повторений.

### **Вариант 4**

1. Создайте новую таблицу и заполните ее номерами деталей, которые поставляются кем-либо из Белграда, либо применяются для какого-то изделия со сборкой в Белграде.

2. Добавить в новую таблицу нового поставщика S10. В поле "фамилия" указать "Уайт". Рейтинг оставить со значением null.

### **Вариант 5**

1. Из базы данных исключить все изделия, производимые в Москве и удалить все поставки в Москву.

2. Создайте новую таблицу и заполните ее городами, в которых расположен как минимум, один поставщик, изделие или деталь.

### **Контрольные вопросы**

1. Что такое коррелированный запрос? Чем отличается коррелированный запрос от некоррелированного?

2. Какие существуют ограничения на формирование коррелированного запроса?

3. Каким образом сохранить результаты запроса в таблице?
4. Какими средствами SQL реализуются следующие операции реляционной алгебры: ограничение, декартово произведение, проекция, пересечение, объединение, разность, соединение?
5. Что такое внешнее соединение?
6. В каких случаях вместо фразы IN можно использовать операцию сравнения?
7. Какие существуют средства группирования в SQL? Как они используются?

## **ЛАБОРАТОРНАЯ РАБОТА № 4-5**

### **Представления. Создание представлений для БД хранимые процедуры и функции, триггеры для работы с БД**

#### **Цель работы**

Познакомиться с возможностями MySQL по работе с хранимыми процедурами, функциями, триггерами, представлениями.

#### **Методические указания к выполнению работы** ***Представления***

Временные таблицы с динамически формируемым содержимым часто принято называть представлением. На текущий момент существует два основных механизма создания представлений: с созданием самостоятельной таблицы либо с применением алгоритма временных таблиц MySQL. Рассмотрим детальнее первый способ, потому что второй способ, скорее всего, был принят из соображений унификации. UPDATE, DELETE к ним применяются точно так же, как и к реально существующим таблицам. Кроме того, неочевидным плюсом является то что, используя представление можно гибко управлять правами среди пользователей базы данных так как, есть

возможность предоставлять доступ на уровне отдельных записей различных таблиц.

### ***Хранимые процедуры и функции***

Сегодня, MySQL научилась создавать и хранить заранее заготовленные функции и процедуры. Основные отличия функций и процедур заключаются в следующем:

- функция возвращает результат, поэтому в заголовке обязательно указывается тип возвращаемого значения;
- функция обязательно должна иметь строку, в которой переменной Result присваивается значение результата. Именно значение в переменной Result является результатом выполнения функции;
- процедура вызывается как отдельный оператор;
- функция может вызываться там, где можно ставить выражение соответствующего типа, например, в правой части оператора присвоения.

Функции, написанные пользователем похожи на уже существующие процедуры. Основное отличие заключено в том, что они имеют меньше возможностей, например, они обязательно должны возвращать всего одно значение (скалярное или табличное). Их основное преимущество – удобное использование с точки зрения синтаксиса.

Оба типа могут возвращать значения, будь оно скалярное или табличное. Основное различие находится в том, что процедура вызывается из отдельной команды, а функция вызывается из запроса.

### ***Триггеры***

Хранимая процедура специального вида получила название триггер. Пользователь может вызывать триггер не как простую функцию или процедуру, а путем наступления некоего указанного действия, которое повлекло за собой событие. Например, событием может быть добавление новой строки оператором INSERT или удаление существующей строки оператором DELETE, либо модификация данных в таблице оператором UPDATE в определенном столбце внутри базы данных. Очень часто триггеры используются для того чтобы обеспечить целостность данных при реализации какой-

либо бизнес-логики. Эти процедуры запускаются сервером в автоматическом режиме при срабатывании события, с которым он связан. Когда наступает событие, срабатывает триггер. Все модификации, которые производит триггер, выполняются в одной транзакции с тем событием, которое его вызвало. Отсюда вытекает то, что при какой-либо ошибке в работе триггера откатывается вся транзакция целиком. С помощью, ключевых слов BEFORE и AFTER можно задать в какой, момент должен отработать триггер – до выполнения, связанного с ним события или после. Если триггер вызывается с параметром BEFORE, то он может, например, изменить модифицируемую событием запись.

Триггеры в некоторых системах управления базами данных накладывают ограничения на содержимое триггера. Например, они могут запретить любые изменения в той таблице, на которой работает триггер.

Аналогично таблице, триггеры могут работать и на представлении (VIEW). Тогда с их помощью можно реализовать механизм “обновляемого представления”. В таком случае, использование ключевых слов (BEFORE, AFTER) влияет в первую очередь, на порядок вызова триггеров. Это обусловлено тем, что событие (обновление, удаление или вставка) фактически не происходит.

## **Задание на работу**

### **Представления**

1. Создайте представление, содержащее объем поставок деталей для изделий за указанный месяц.

2. В таблицу SPJ добавьте новый столбец “стоимость детали” (ALTER TABLE). Для этой таблицы создайте представление с полями: наименование поставщика, наименование детали, наименование изделия, стоимость детали, количество, стоимость поставки.

3. В таблицу P добавьте столбец “стоимость детали” (ALTER TABLE). Придумайте представление в которой содержится стоимость поставки.

## Процедуры

1. Напишите процедуру, которая будет возвращать детали, из которых состоит изделие.

2. Напишите процедуру, которая будет считать возвращаемую расчетную стоимость с учетом того, что для изделия необходимо иметь ровно  $K$  деталей каждого наименования. В таблицах поставщиков, деталей изделий и поставок они приведены.

3. Напишите процедуру, выполняющую вычисление веса изделия с учетом того, что для каждого изделия нужно  $K$  деталей каждого типа. Данные приведены в таблицах поставщиков, деталей изделий и поставок.

4. Выполнить ABC анализ (разделение поставщиков по объемам поставки 20, 40, 60%) используя условные операторы.

5. Выполнить ABC анализ (разделение поставщиков по стоимости поставки 20, 40, 60%) используя условные операторы.

6. Вычислите поставщика для изделий из таблиц поставщиков, деталей изделий и поставок для оптимизации производства по критерию максимального количества изделий за период.

7. Вычислите поставщика для изделий из таблиц поставщиков, деталей изделий и поставок для оптимизации производства по критерию максимального количества изделий по минимальной цене.

8. Вычислите поставщика для изделий из таблиц поставщиков, деталей изделий и поставок для оптимизации производства по критерию максимального количества изделий по минимальной цене с учетом того, что один поставщик может сделать не более одной поставки за неделю, причем объем поставки – не более среднего за период.

## Функции

1. Написать функцию, которая отдает таблицу, содержащую перечень изделий, для которых выполняется поставка.

|    |          |
|----|----------|
| S1 | J1 J2 J4 |
| S2 | J5       |

2. Написать функцию, которая отдает таблицу, содержащую наименования изделий, для которых выполняется поставка

3. Написать функцию, которая отдает таблицу, в которой представлен перечень деталей дневной поставки

4. Написать функцию, которая отдает таблицу, в которой представлен перечень наименований деталей дневной поставки

5. Написать функцию, отдающую наименование поставщика, который поставяет деталей больше остальных

6. Написать функцию, возвращающую наименование поставщика, который поставяет для конкретного изделия больше, чем остальные

## Работа с текстовым файлом

1. Используя автоматизированные средства, создайте текстовый файл, в котором указана информация о поставщике, поставившем за месяц деталей на большую сумму и меньший вес.

## Курсоры

1. Сделать так, чтобы дата поставки автоматически проставлялась текущей.

## Контрольные вопросы

1. Что такое представление? Особенности создания представлений.

2. Дайте определение хранимым процедурам. Каковы правила их применения?
3. Дайте определение функции. Каким правилам они подчинены?
4. Что такое триггеры? Как они работают в БД?

## **ЛАБОРАТОРНАЯ РАБОТА № 6**

### **Работа с внешними базами данных. Настройка ограничения доступа**

#### **Цель работы**

Ознакомиться со средствами предоставления полномочий на использование баз данных и таблиц и основами работы с внешними базами данных.

#### **Методические указания к выполнению работы**

##### **Предоставление доступа к базам данных**

В современных базах данных существует специальная база данных, которая используется для предоставления доступа. Механизм использует права. Права могут быть основаны на именах серверов и/или на именах пользователей. Права могут предоставляться для одной или нескольких баз данных.

Учетные записи пользователей в этой базе снабжены паролями. В целях безопасности пароли шифруются. Этот механизм направлен на то, чтобы исключить перехват паролей посторонними.

В системе управления базами данных MySQL для этого организованы три таблицы:

База данных: mysql Таблица: db

| База данных: mysql Таблица: db |           |      |      |           |       |
|--------------------------------|-----------|------|------|-----------|-------|
| Поле                           | Тип       | Null | Ключ | Умолчание | Extra |
| Хост                           | char (60) |      | PRI  |           |       |
| Db                             | char (32) |      | PRI  |           |       |

|              |           |  |     |   |  |
|--------------|-----------|--|-----|---|--|
| Пользователь | char (16) |  | PRI |   |  |
| Select_priv  | char (1)  |  |     | N |  |
| Insert_priv  | char (1)  |  |     | N |  |
| Update_priv  | char (1)  |  |     | N |  |
| Priv_delete  | char (1)  |  |     | N |  |
| Priv_create  | char (1)  |  |     | N |  |
| Priv_drop    | char (1)  |  |     | N |  |

База данных: mysql Таблица: host

| Поле        | Тип       | Null | Ключ | Умолчание | Extra |
|-------------|-----------|------|------|-----------|-------|
| Хост        | char (60) |      | PRI  |           |       |
| Db          | char (32) |      | PRI  |           |       |
| Select_priv | char (1)  |      |      | N         |       |
| Insert_priv | char (1)  |      |      | N         |       |
| Update_priv | char (1)  |      |      | N         |       |
| Priv_delete | char (1)  |      |      | N         |       |
| Priv_create | char (1)  |      |      | N         |       |
| Priv_drop   | char (1)  |      |      | N         |       |

База данных: mysql Таблица: user

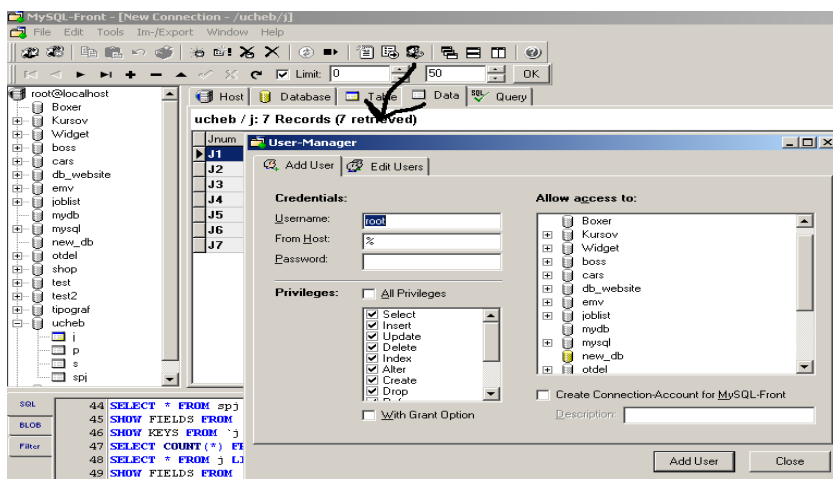
| Поле         | Тип       | Null | Key | Умолчание | Extra |
|--------------|-----------|------|-----|-----------|-------|
| Хост         | char (60) |      | PRI |           |       |
| Пользователь | char (16) |      | PRI |           |       |
| Пароль       | char (8)  |      |     |           |       |
| Select_priv  | char (1)  |      |     | N         |       |
| Insert_priv  | char (1)  |      |     | N         |       |

|               |          |  |  |   |  |
|---------------|----------|--|--|---|--|
| Update_priv   | char (1) |  |  | N |  |
| Priv_delete   | char (1) |  |  | N |  |
| Priv_create   | char (1) |  |  | N |  |
| Priv_drop     | char (1) |  |  | N |  |
| Priv_reload   | char (1) |  |  | N |  |
| Priv_shutdown | char (1) |  |  | N |  |
| Priv_proc     | char (1) |  |  | N |  |
| Priv_file     | char (1) |  |  | N |  |

База данных, которую открыли с помощью операторов use Database или с помощью утилиты mysqladmin называют текущей. Все остальные базы данных принято называть внешними. Если нужно обратиться к таблице во внешней базе данных, то указывают имя этой базы в составе имени таблицы. К примеру, salesdb:contracts. Тут salesdb – название внешней базы данных, в которой расположена таблица contracts. К этой строке иногда добавляют еще и имя сервера, то есть, имя физической машины, на которой работает сервер базы данных MySQL. Итого, полный путь к таблице, расположенной на сервере central, в базе данных salesdb с именем contracts выглядит так: salesdb@central:contracts.

Кроме того, в таких программах, как MYSQL-FRONT также реализован подход, дающий возможность наделять пользователей правами. Этот процесс показан на рисунке.

Когда соединение с сервером будет установлено необходимо, выполнить последовательность, описанную в лабораторной работе. Если соединение отсутствует, вам необходимо завести нового пользователя внутри базы данных, наделить его необходимыми привилегиями.



**Рис. Редактирование прав пользователя**

### **Задание на работу**

1. Откройте таблицу поставщиков S. Найдите записи с Вашими фамилиями. При отсутствии – создайте записи.

2. Откройте форму, сделанную в третьей лабораторной работе, когда выполнялись действия с таблицей поставок SPJ. Обеспечьте работу ввода и модификации данных. Добавьте несколько строк, но не менее 5, связанных с поставками. Привяжите их к Вашим фамилиям.

3. Выберите вариант и согласно нему выполните два запроса к базе данных. Вам необходимо чтобы данные из запроса выбирались из таблиц именно Вашей базы данных.

4. Повторите выполнения задания из пункта 3 таким образом, чтобы данные о номенклатуре деталей и изделий (Р, Л) были из Вашей базы данных, а сведения о поставщиках и поставках (S и SPJ) брались из базы данных соседней команды. Чтобы все получилось и не возникло проблем нужно узнать параметры подключения у соседей. Если сеть отсутствует, то убедитесь в невозможности выполнить задание.

5. Попросите Ваших товарищей, чтобы те предоставили Вам все необходимые права на просмотр используемых

Вами таблиц. При необходимости завели для вас нового пользователя.

6. Повторите выполнение действий, описанные в пункте 4, сравните с результатами из пункта 3.

7. Попробуйте изменить какую-либо информацию в таблице S у ваших товарищей. Докажите, что это сделать невозможно, пока вам выданы права только на чтение. Сделайте вывод.

8. Попросите у Ваших товарищей права на изменение данных в базе данных.

9. Повторите выполнение пункта 7. Успешно ли выполнение? Сделайте вывод о проделанной работе.

10. После того как Ваши товарищи так же дошли до этого этапа попытайтесь удалить из таблицы S внешней базы данных всех поставщиков с именами, которые относятся к вашим товарищам – владельцам внешней базы данных. Кроме того, удалите связанные с ним поставки, хранящиеся в таблице SPJ. Докажите, что выполнение задания невозможно.

11. Попросите у Ваших товарищей права на удаление данных в базе данных.

12. Повторите выполнение пункта 10. Успешно ли выполнение? Сделайте вывод о проделанной работе.

13. Заберите все выданные ранее права на использование Вашей базы.

### **Индивидуальные варианты заданий**

#### **Вариант 1**

1. Вывести все изделия, использующие как минимум одну деталь, которую поставляет поставщик S6.

#### **Вариант 2**

1. Вывести все цвета деталей, поставляемые поставщиком S6.

#### **Вариант 3**

1. Вывести все изделия, детали для которых поставляет поставщик S6.

#### Вариант 4

1. Выбрать те изделия, детали для которых поставяет только поставщик S6.

#### Вариант 5

1. Выбрать те изделия, детали для которых поставяет только поставщик S6.

### **Контрольные вопросы**

1. Кого можно считать владельцем базы данных?
2. Какими правами обладают другие пользователи по отношению к Вашей базе данных?
3. Какими правами обладает администратор базы данных по отношению к Вашей базе данных?
4. Каким образом предоставляются права на пользование базой данных и отдельными ее таблицами?
5. Каким образом изымаются права на пользование базой данных и отдельными ее таблицами?
6. Что такое внешняя база данных?
7. Как идентифицируется таблица внешней базы данных?
8. Как идентифицируется таблица внешней распределенной базы данных?

## ЗАКЛЮЧЕНИЕ

Возможности современных информационных технологий с каждым годом становятся все более широкими и затрагивают все больше сфер жизни человека. Глобальными информационными системами пользуются все больше и в самых различных сферах жизни. Работа с моделями представления данных ведется на всех этапах проектирования и разработки информационных систем. На этапе проектирования ведется разработка интегрированной модели системы, которая включает в себя разработку как концептуальных, так и физических моделей. Проектирование таких сложных моделей становится все более трудной и важной задачей.

В учебном пособии описываются вопросы, которые касаются администрирования систем управления базами данных. В пособии также есть список общих обязанностей администратора и инструкции по их выполнению.

Материалы, представленные в данном пособии, помогут найти большое количество решений, которые необходимо знать при работе с MySQL, и которые помогут при работе с данными.

## БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. MySQL: учебное пособие: пер. с англ. – М.: Издательский дом «Вильямс», 2005. – 304 с.: ил.
2. Кузнецов, М. В. MySQL 5 / М. В. Кузнецов, И. В. Симдянов. – СПб.: БХВ-Петербург, 2010. – 1024 с.: ил.
3. Гольцман, В. MySQL 5.0 / В. Гольцман – СПб.: Питер, 2010. – 253 с.: ил.
4. Шварц, Б. Зайцев П., Ткаченко В. / MySQL по максимуму. Оптимизация, репликация, резервное копирование/ Б. Шварц, П. Зайцев, В. Ткаченко. 3-е изд. – СПб.: Питер, 2018. – 864 с.: ил.

**Сгенерированный SQL код**

```
CREATE TABLE Compliance (  
    NumberCompliance    int NOT NULL,  
    NumberShop           int NULL,  
    NumberEquipment      int NULL,  
    NumberDocuments      int NULL  
)  
go
```

```
ALTER TABLE Compliance  
    ADD PRIMARY KEY NONCLUSTERED (NumberCompliance)  
go
```

```
CREATE TABLE Coordination (  
    NumberDocuments      int NULL,  
    nomerOfEmployees     int NULL,  
    NumberCoordination   int NULL,  
    Coordination_type     varchar (200) NULL  
)  
go
```

```
ALTER TABLE Coordination  
    ADD PRIMARY KEY NONCLUSTERED (NumberCoordination)  
go
```

```
CREATE TABLE Counters (  
    Number_counters      int NULL,  
    Datetime_counters    datetime NULL,  
    indication_counters  int NULL,  
    N_counters           int NOT NULL,  
    NumberEquipment      int NULL  
)  
go
```

```
ALTER TABLE Counters
    ADD PRIMARY KEY NONCLUSTERED (N_counters)
go
```

```
CREATE TABLE Documents (
    Date            datetime NOT NULL,
    NumberDocuments int NOT NULL,
    Text            varchar(200) NULL,
    NumberOfOrganization int NULL,
    nomerOfEmployees int NULL,
    Type_Documents  varchar(200) NULL,
    State           varchar(200) NULL,
    NumberShop      int NULL,
    Number_of_TO    int NULL,
    NumberEquipment int NULL
)
go
```

```
ALTER TABLE Documents
    ADD PRIMARY KEY NONCLUSTERED (NumberDocuments)
go
```

```
CREATE TABLE Employees (
    NumberOfOrganization int NULL,
    Surname              varchar (20) NULL,
    name                 varchar (20) NULL,
    nomerOfEmployees    int NOT NULL,
    middleName           varchar (20) NULL,
    address              varchar (200) NULL,
    Status               varchar (200) NULL,
    Fone                 int NULL,
    Login                varchar(200) NULL,
    Password             varchar(200) NULL
)
```

```
)  
go
```

```
ALTER TABLE Employees  
    ADD PRIMARY KEY NONCLUSTERED (nomerOfEm-  
ployees)  
go
```

```
CREATE TABLE Equipment (  
    Name          varchar(200) NULL,  
    Unit_of_measure  varchar(200) NULL,  
    NumberEquipment  int NOT NULL,  
    Cost           int NULL,  
    Brand          varchar(200) NULL,  
    Code          varchar(200) NULL,  
    Manufacturer    varchar(200) NULL,  
    Frequency_of_checking datetime NULL,  
    NumberShop      int NULL  
)  
go
```

```
ALTER TABLE Equipment  
    ADD PRIMARY KEY NONCLUSTERED (NumberEquip-  
ment)  
go
```

```
CREATE TABLE Materials (  
    Name_materials  char(18) NULL,  
    quantity_materials int NULL,  
    Vid_material    char(18) NULL,  
    Material_manufacturer varchar(200) NULL,  
    Number_materials int NOT NULL  
)
```

```
go
```

```
ALTER TABLE Materials
    ADD PRIMARY KEY NONCLUSTERED (Number_materials)
go
```

```
CREATE TABLE Organization (
    name          varchar(200) NULL,
    NumberOfOrganization int NOT NULL,
    address       varchar(200) NULL,
    INN           int NULL,
    Phone         int NULL
)
go
```

```
ALTER TABLE Organization
    ADD PRIMARY KEY NONCLUSTERED (NumberOfOrganization)
go
```

```
CREATE TABLE Schedule_of_TI (
    Date          datetime NULL,
    Number_of_TO  int NULL,
    Type          varchar(200) NULL,
    NumberEquipment int NULL
)
go
```

```
ALTER TABLE Schedule_of_TI
    ADD PRIMARY KEY NONCLUSTERED (Number_of_TO)
go
```

```
CREATE TABLE Shop (
    Name_Shop      varchar(200) NULL,
    NumberShop     int NOT NULL
)
go
```

```
ALTER TABLE Shop
    ADD PRIMARY KEY NONCLUSTERED (NumberShop)
go
```

```
CREATE TABLE Work_Log (
    Name_Work_Log   char(18) NULL,
    Date_time       datetime NULL,
    Number_Work_Log int NOT NULL,
    type_of_work    varchar(200) NULL,
    malfunction     varchar(200) NULL,
    NumberEquipment int NULL,
    Number_materials int NULL,
    quantity_materials int NULL,
    nomerOfEmployees int NULL
)
go
```

```
ALTER TABLE Work_Log
    ADD PRIMARY KEY NONCLUSTERED (Number_Work_Log)
go
```

```
ALTER TABLE Compliance
    ADD FOREIGN KEY (NumberDocuments)
        REFERENCES Documents
```

go

```
ALTER TABLE Compliance
  ADD FOREIGN KEY (NumberEquipment)
    REFERENCES Equipment
```

go

```
ALTER TABLE Compliance
  ADD FOREIGN KEY (NumberShop)
    REFERENCES Shop
```

go

```
ALTER TABLE Coordination
  ADD FOREIGN KEY (nomerOfEmployees)
    REFERENCES Employees
```

go

```
ALTER TABLE Coordination
  ADD FOREIGN KEY (NumberDocuments)
    REFERENCES Documents
```

go

```
ALTER TABLE Counters
  ADD FOREIGN KEY (NumberEquipment)
    REFERENCES Equipment
```

go

```
ALTER TABLE Documents
  ADD FOREIGN KEY (NumberEquipment)
    REFERENCES Equipment
```

go

```
ALTER TABLE Documents
    ADD FOREIGN KEY (Number_of_TO)
        REFERENCES Schedule_of_TI
go
```

```
ALTER TABLE Documents
    ADD FOREIGN KEY (NumberShop)
        REFERENCES Shop
go
```

```
ALTER TABLE Documents
    ADD FOREIGN KEY (nomerOfEmployees)
        REFERENCES Employees
go
```

```
ALTER TABLE Documents
    ADD FOREIGN KEY (NumberOfOrganization)
        REFERENCES Organization
go
```

```
ALTER TABLE Employees
    ADD FOREIGN KEY (NumberOfOrganization)
        REFERENCES Organization
go
```

```
ALTER TABLE Equipment
    ADD FOREIGN KEY (NumberShop)
        REFERENCES Shop
go
```

```
ALTER TABLE Schedule_of_TI
    ADD FOREIGN KEY (NumberEquipment)
        REFERENCES Equipment
go
```

```
ALTER TABLE Work_Log
    ADD FOREIGN KEY (nomerOfEmployees)
        REFERENCES Employees
go
```

```
ALTER TABLE Work_Log
    ADD FOREIGN KEY (Number_materials)
        REFERENCES Materials
go
```

```
ALTER TABLE Work_Log
    ADD FOREIGN KEY (NumberEquipment)
        REFERENCES Equipment
go
```

```
create trigger tI_Compliance on Compliance for INSERT as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Compliance */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @errno int,
            @errmsg varchar(255)
```

```

select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Documents R/40 Compliance ON CHILD INSERT SET
NULL */
if
/* update(NumberDocuments) */
update(NumberDocuments)
begin
update Compliance
set
/* Compliance.NumberDocuments = NULL */
Compliance.NumberDocuments = NULL
from Compliance,inserted
where
/* */
and
not exists (
select * from Documents
where
/* inserted.NumberDocuments = Docu-
ments.NumberDocuments */
inserted.NumberDocuments = Docu-
ments.NumberDocuments
)
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/39 Compliance ON CHILD INSERT SET NULL
*/
if
/* update(NumberEquipment) */
update(NumberEquipment)
begin
update Compliance
set
/* Compliance.NumberEquipment = NULL */
Compliance.NumberEquipment = NULL

```

```

from Compliance,inserted
where
  /* */
  and
  not exists (
    select * from Equipment
    where
      /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
      inserted.NumberEquipment = Equip-
ment.NumberEquipment
  )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/38 Compliance ON CHILD INSERT SET NULL */
if
  /* update(NumberShop) */
  update(NumberShop)
begin
  update Compliance
  set
    /* Compliance.NumberShop = NULL */
    Compliance.NumberShop = NULL
  from Compliance,inserted
  where
    /* */
    and
    not exists (
      select * from Shop
      where
        /* inserted.NumberShop = Shop.NumberShop */
        inserted.NumberShop = Shop.NumberShop
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

create trigger tU_Compliance on Compliance for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Compliance */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @insNumberCompliance int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Documents R/40 Compliance ON CHILD UPDATE SET
    NULL */
    if
        /* update(NumberDocuments) */
        update(NumberDocuments)
    begin
        update Compliance
        set
            /* Compliance.NumberDocuments = NULL */
            Compliance.NumberDocuments = NULL
        from Compliance,inserted
        where
            /* */
            and
            not exists (
                select * from Documents

```

```

        where
        /* inserted.NumberDocuments = Docu-
ments.NumberDocuments */
        inserted.NumberDocuments = Docu-
ments.NumberDocuments
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/39 Compliance ON CHILD UPDATE SET
NULL */
if
    /* update(NumberEquipment) */
    update(NumberEquipment)
begin
    update Compliance
    set
        /* Compliance.NumberEquipment = NULL */
        Compliance.NumberEquipment = NULL
    from Compliance,inserted
    where
        /* */
        and
        not exists (
            select * from Equipment
            where
                /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
                inserted.NumberEquipment = Equip-
ment.NumberEquipment
        )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/38 Compliance ON CHILD UPDATE SET NULL */
if
    /* update(NumberShop) */

```

```

update(NumberShop)
begin
  update Compliance
  set
    /* Compliance.NumberShop = NULL */
    Compliance.NumberShop = NULL
  from Compliance,inserted
  where
    /* */
    and
    not exists (
      select * from Shop
      where
        /* inserted.NumberShop = Shop.NumberShop */
        inserted.NumberShop = Shop.NumberShop
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tI_Coordination on Coordination for INSERT as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Coordination */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @errno int,
          @errmsg varchar(255)

```

```

select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Employees R/32 Coordination ON CHILD INSERT SET
NULL */
if
/* update(nomerOfEmployees) */
update(nomerOfEmployees)
begin
update Coordination
set
/* Coordination.nomerOfEmployees = NULL */
Coordination.nomerOfEmployees = NULL
from Coordination,inserted
where
/* */
and
not exists (
select * from Employees
where
/* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees
)
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Documents R/31 Coordination ON CHILD INSERT SET
NULL */
if
/* update(NumberDocuments) */
update(NumberDocuments)
begin
update Coordination
set
/* Coordination.NumberDocuments = NULL */
Coordination.NumberDocuments = NULL

```

```

from Coordination,inserted
where
  /* */
  and
  not exists (
    select * from Documents
    where
      /* inserted.NumberDocuments = Docu-
ments.NumberDocuments */
      inserted.NumberDocuments = Docu-
ments.NumberDocuments
  )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tU_Coordination on Coordination for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Coordination */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @insNumberCoordination int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */

```

```

/* Employees R/32 Coordination ON CHILD UPDATE SET
NULL */
if
  /* update(nomerOfEmployees) */
  update(nomerOfEmployees)
begin
  update Coordination
  set
    /* Coordination.nomerOfEmployees = NULL */
    Coordination.nomerOfEmployees = NULL
  from Coordination,inserted
  where
    /* */
    and
    not exists (
      select * from Employees
      where
        /* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
        inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees
    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Documents R/31 Coordination ON CHILD UPDATE SET
NULL */
if
  /* update(NumberDocuments) */
  update(NumberDocuments)
begin
  update Coordination
  set
    /* Coordination.NumberDocuments = NULL */
    Coordination.NumberDocuments = NULL
  from Coordination,inserted
  where

```

```

/* */
and
not exists (
  select * from Documents
  where
    /* inserted.NumberDocuments = Docu-
ments.NumberDocuments */
    inserted.NumberDocuments = Docu-
ments.NumberDocuments
)
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tI_Counters on Counters for INSERT as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Counters */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/52 Counters ON CHILD INSERT SET NULL */
if
  /* update(NumberEquipment) */
  update(NumberEquipment)

```

```

begin
  update Counters
  set
    /* Counters.NumberEquipment = NULL */
    Counters.NumberEquipment = NULL
  from Counters,inserted
  where
    /* */
    and
    not exists (
      select * from Equipment
      where
        /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
        inserted.NumberEquipment = Equip-
ment.NumberEquipment
    )
  end

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

create trigger tU_Counters on Counters for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Counters */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @insN_counters int,
          @errno int,
          @errmsg varchar(255)

```

```

select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/52 Counters ON CHILD UPDATE SET NULL
*/
if
  /* update(NumberEquipment) */
  update(NumberEquipment)
begin
  update Counters
  set
    /* Counters.NumberEquipment = NULL */
    Counters.NumberEquipment = NULL
  from Counters,inserted
  where
    /* */
    and
    not exists (
      select * from Equipment
      where
        /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
        inserted.NumberEquipment = Equip-
ment.NumberEquipment
    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

create trigger tD\_Documents on Documents for DELETE as

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Documents */
begin
  declare @errno int,
          @errmsg varchar(255)
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Documents R/40 Compliance ON PARENT DELETE SET
  NULL */
  update Compliance
  set
    /* Compliance.NumberDocuments = NULL */
    Compliance.NumberDocuments = NULL
  from Compliance,deleted
  where
    /* Compliance.NumberDocuments = delet-
ed.NumberDocuments */
    Compliance.NumberDocuments = delet-
ed.NumberDocuments

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Documents R/31 Coordination ON PARENT DELETE SET
  NULL */
  update Coordination
  set
    /* Coordination.NumberDocuments = NULL */
    Coordination.NumberDocuments = NULL
  from Coordination,deleted
  where
    /* Coordination.NumberDocuments = delet-
ed.NumberDocuments */
    Coordination.NumberDocuments = delet-
ed.NumberDocuments

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  return
error:

```

```

        raiserror @errno @errmsg
        rollback transaction
end
go

create trigger tI_Documents on Documents for INSERT as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Documents */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Equipment R/45 Documents ON CHILD INSERT SET NULL
*/
    if
        /* update(NumberEquipment) */
        update(NumberEquipment)
    begin
        update Documents
        set
            /* Documents.NumberEquipment = NULL */
            Documents.NumberEquipment = NULL
        from Documents,inserted
        where
            /* */
            and
            not exists (
                select * from Equipment
                where
                    /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */

```

```

        inserted.NumberEquipment = Equip-
ment.NumberEquipment
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Schedule_of_TI R/44 Documents ON CHILD INSERT SET
NULL */

```

```

if
    /* update(Number_of_TO) */
    update(Number_of_TO)
begin
    update Documents
    set
        /* Documents.Number_of_TO = NULL */
        Documents.Number_of_TO = NULL
    from Documents,inserted
    where
        /* */
        and
        not exists (
            select * from Schedule_of_TI
            where
                /* inserted.Number_of_TO = Sched-
ule_of_TI.Number_of_TO */
                inserted.Number_of_TO = Sched-
ule_of_TI.Number_of_TO
        )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/43 Documents ON CHILD INSERT SET NULL */
if
    /* update(NumberShop) */
    update(NumberShop)
begin
    update Documents

```

```

set
/* Documents.NumberShop = NULL */
Documents.NumberShop = NULL
from Documents,inserted
where
/* */
and
not exists (
select * from Shop
where
/* inserted.NumberShop = Shop.NumberShop */
inserted.NumberShop = Shop.NumberShop
)
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Employees R/21 Documents ON CHILD INSERT SET NULL
*/
if
/* update(nomerOfEmployees) */
update(nomerOfEmployees)
begin
update Documents
set
/* Documents.nomerOfEmployees = NULL */
Documents.nomerOfEmployees = NULL
from Documents,inserted
where
/* */
and
not exists (
select * from Employees
where
/* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees

```

```

    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Organization R/20 Documents ON CHILD INSERT SET
NULL */
if
  /* update(NumberOfOrganization) */
  update(NumberOfOrganization)
begin
  update Documents
  set
    /* Documents.NumberOfOrganization = NULL */
    Documents.NumberOfOrganization = NULL
  from Documents,inserted
  where
    /* */
    and
    not exists (
      select * from Organization
      where
        /* inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization */
        inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization
    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tU_Documents on Documents for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Documents */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @insNumberDocuments int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Documents R/40 Compliance ON PARENT UPDATE SET
  NULL */
  if
    /* update(NumberDocuments) */
    update(NumberDocuments)
  begin
    update Compliance
    set
      /* Compliance.NumberDocuments = NULL */
      Compliance.NumberDocuments = NULL
    from Compliance,deleted
    where
      /* Compliance.NumberDocuments = delet-
ed.NumberDocuments */
      Compliance.NumberDocuments = delet-
ed.NumberDocuments
  end

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Documents R/31 Coordination ON PARENT UPDATE SET
  NULL */
  if
    /* update(NumberDocuments) */
    update(NumberDocuments)

```

```

begin
  update Coordination
  set
    /* Coordination.NumberDocuments = NULL */
    Coordination.NumberDocuments = NULL
  from Coordination,deleted
  where
    /* Coordination.NumberDocuments = delet-
ed.NumberDocuments */
    Coordination.NumberDocuments = delet-
ed.NumberDocuments
  end

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Equipment R/45 Documents ON CHILD UPDATE SET
NULL */
  if
    /* update(NumberEquipment) */
    update(NumberEquipment)
  begin
    update Documents
    set
      /* Documents.NumberEquipment = NULL */
      Documents.NumberEquipment = NULL
    from Documents,inserted
    where
      /* */
      and
      not exists (
        select * from Equipment
        where
          /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
          inserted.NumberEquipment = Equip-
ment.NumberEquipment
      )
  end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Schedule_of_TI R/44 Documents ON CHILD UPDATE SET
NULL */
if
  /* update(Number_of_TO) */
  update(Number_of_TO)
begin
  update Documents
  set
    /* Documents.Number_of_TO = NULL */
    Documents.Number_of_TO = NULL
  from Documents,inserted
  where
    /* */
    and
    not exists (
      select * from Schedule_of_TI
      where
        /* inserted.Number_of_TO = Sched-
ule_of_TI.Number_of_TO */
        inserted.Number_of_TO = Sched-
ule_of_TI.Number_of_TO
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/43 Documents ON CHILD UPDATE SET NULL */
if
  /* update(NumberShop) */
  update(NumberShop)
begin
  update Documents
  set
    /* Documents.NumberShop = NULL */
    Documents.NumberShop = NULL
  from Documents,inserted

```

```

where
/* */
and
not exists (
  select * from Shop
  where
    /* inserted.NumberShop = Shop.NumberShop */
    inserted.NumberShop = Shop.NumberShop
)
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Employees R/21 Documents ON CHILD UPDATE SET
NULL */
if
  /* update(nomerOfEmployees) */
  update(nomerOfEmployees)
begin
  update Documents
  set
    /* Documents.nomerOfEmployees = NULL */
    Documents.nomerOfEmployees = NULL
  from Documents,inserted
  where
    /* */
    and
    not exists (
      select * from Employees
      where
        /* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
        inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees
    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */

```

```

/* Organization R/20 Documents ON CHILD UPDATE SET
NULL */
if
/* update(NumberOfOrganization) */
update(NumberOfOrganization)
begin
update Documents
set
/* Documents.NumberOfOrganization = NULL */
Documents.NumberOfOrganization = NULL
from Documents,inserted
where
/* */
and
not exists (
select * from Organization
where
/* inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization */
inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization
)
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
raiserror @errno @errmsg
rollback transaction
end
go

create trigger tD_Employees on Employees for DELETE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Employees */
begin

```

```

declare @errno int,
        @errmsg varchar(255)
/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* Employees R/51 Work_Log ON PARENT DELETE SET
NULL */
update Work_Log
set
    /* Work_Log.nomerOfEmployees = NULL */
    Work_Log.nomerOfEmployees = NULL
from Work_Log,deleted
where
    /* Work_Log.nomerOfEmployees = delet-
ed.nomerOfEmployees */
    Work_Log.nomerOfEmployees = delet-
ed.nomerOfEmployees

/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* Employees R/32 Coordination ON PARENT DELETE SET
NULL */
update Coordination
set
    /* Coordination.nomerOfEmployees = NULL */
    Coordination.nomerOfEmployees = NULL
from Coordination,deleted
where
    /* Coordination.nomerOfEmployees = delet-
ed.nomerOfEmployees */
    Coordination.nomerOfEmployees = delet-
ed.nomerOfEmployees

/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* Employees R/21 Documents ON PARENT DELETE SET
NULL */
update Documents
set
    /* Documents.nomerOfEmployees = NULL */
    Documents.nomerOfEmployees = NULL

```

```

    from Documents,deleted
    where
        /* Documents.nomerOfEmployees = delet-
ed.nomerOfEmployees */
        Documents.nomerOfEmployees = delet-
ed.nomerOfEmployees

    /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
    return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

create trigger tI_Employees on Employees for INSERT as
/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Employees */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
    /* Organization R/5 Employees ON CHILD INSERT SET NULL
*/
    if
        /* update(NumberOfOrganization) */
        update(NumberOfOrganization)
    begin
        update Employees
        set
            /* Employees.NumberOfOrganization = NULL */

```

```

    Employees.NumberOfOrganization = NULL
from Employees,inserted
where
    /* */
    and
    not exists (
        select * from Organization
        where
            /* inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization */
            inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tU_Employees on Employees for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Employees */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @insnomerOfEmployees int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */

```

```
/* Employees R/51 Work_Log ON PARENT UPDATE SET  
NULL */
```

```
if
```

```
/* update(nomerOfEmployees) */
```

```
update(nomerOfEmployees)
```

```
begin
```

```
update Work_Log
```

```
set
```

```
/* Work_Log.nomerOfEmployees = NULL */
```

```
Work_Log.nomerOfEmployees = NULL
```

```
from Work_Log,deleted
```

```
where
```

```
/* Work_Log.nomerOfEmployees = delet-
```

```
ed.nomerOfEmployees */
```

```
Work_Log.nomerOfEmployees = delet-
```

```
ed.nomerOfEmployees
```

```
end
```

```
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
```

```
/* Employees R/32 Coordination ON PARENT UPDATE SET  
NULL */
```

```
if
```

```
/* update(nomerOfEmployees) */
```

```
update(nomerOfEmployees)
```

```
begin
```

```
update Coordination
```

```
set
```

```
/* Coordination.nomerOfEmployees = NULL */
```

```
Coordination.nomerOfEmployees = NULL
```

```
from Coordination,deleted
```

```
where
```

```
/* Coordination.nomerOfEmployees = delet-
```

```
ed.nomerOfEmployees */
```

```
Coordination.nomerOfEmployees = delet-
```

```
ed.nomerOfEmployees
```

```
end
```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Employees R/21 Documents ON PARENT UPDATE SET
NULL */
if
  /* update(nomerOfEmployees) */
  update(nomerOfEmployees)
begin
  update Documents
  set
    /* Documents.nomerOfEmployees = NULL */
    Documents.nomerOfEmployees = NULL
  from Documents,deleted
  where
    /* Documents.nomerOfEmployees = delet-
ed.nomerOfEmployees */
    Documents.nomerOfEmployees = delet-
ed.nomerOfEmployees
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Organization R/5 Employees ON CHILD UPDATE SET
NULL */
if
  /* update(NumberOfOrganization) */
  update(NumberOfOrganization)
begin
  update Employees
  set
    /* Employees.NumberOfOrganization = NULL */
    Employees.NumberOfOrganization = NULL
  from Employees,inserted
  where
    /* */
    and
    not exists (
      select * from Organization
      where

```

```

        /* inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization */
        inserted.NumberOfOrganization = Organiza-
tion.NumberOfOrganization
    )
end

```

```

/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
return
error:
    raiseerror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tD_Equipment on Equipment for DELETE as
/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Equipment */
begin
    declare @errno int,
            @errmsg varchar(255)
    /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
    /* Equipment R/52 Counters ON PARENT DELETE SET
NULL */
    update Counters
    set
        /* Counters.NumberEquipment = NULL */
        Counters.NumberEquipment = NULL
    from Counters,deleted
    where
        /* Counters.NumberEquipment = deleted.NumberEquipment
*/
        Counters.NumberEquipment = deleted.NumberEquipment

    /* ERwin Bultin Sat Apr 20 22:19:41 2019 */

```

```

/* Equipment R/49 Work_Log ON PARENT DELETE SET
NULL */
update Work_Log
set
/* Work_Log.NumberEquipment = NULL */
Work_Log.NumberEquipment = NULL
from Work_Log,deleted
where
/* Work_Log.NumberEquipment = delet-
ed.NumberEquipment */
Work_Log.NumberEquipment = deleted.NumberEquipment

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/48 Schedule_of_TI ON PARENT DELETE
SET NULL */
update Schedule_of_TI
set
/* Schedule_of_TI.NumberEquipment = NULL */
Schedule_of_TI.NumberEquipment = NULL
from Schedule_of_TI,deleted
where
/* Schedule_of_TI.NumberEquipment = delet-
ed.NumberEquipment */
Schedule_of_TI.NumberEquipment = delet-
ed.NumberEquipment

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/45 Documents ON PARENT DELETE SET
NULL */
update Documents
set
/* Documents.NumberEquipment = NULL */
Documents.NumberEquipment = NULL
from Documents,deleted
where
/* Documents.NumberEquipment = delet-
ed.NumberEquipment */

```

Documents.NumberEquipment = deleted.NumberEquipment

/\* ERwin Bultin Sat Apr 20 22:19:41 2019 \*/

/\* Equipment R/39 Compliance ON PARENT DELETE SET  
NULL \*/

update Compliance

set

/\* Compliance.NumberEquipment = NULL \*/

Compliance.NumberEquipment = NULL

from Compliance,deleted

where

/\* Compliance.NumberEquipment = delet-  
ed.NumberEquipment \*/

Compliance.NumberEquipment = deleted.NumberEquipment

/\* ERwin Bultin Sat Apr 20 22:19:41 2019 \*/

return

error:

raiserror @errno @errmsg

rollback transaction

end

go

create trigger tI\_Equipment on Equipment for INSERT as

/\* ERwin Bultin Sat Apr 20 22:19:41 2019 \*/

/\* INSERT trigger on Equipment \*/

begin

declare @numrows int,

@nullcnt int,

@validcnt int,

@errno int,

@errmsg varchar(255)

select @numrows = @@rowcount

/\* ERwin Bultin Sat Apr 20 22:19:41 2019 \*/

/\* Shop R/47 Equipment ON CHILD INSERT SET NULL \*/

```

if
/* update(NumberShop) */
update(NumberShop)
begin
update Equipment
set
/* Equipment.NumberShop = NULL */
Equipment.NumberShop = NULL
from Equipment,inserted
where
/* */
and
not exists (
select * from Shop
where
/* inserted.NumberShop = Shop.NumberShop */
inserted.NumberShop = Shop.NumberShop
)
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
raiseerror @errno @errmsg
rollback transaction
end
go

```

```

create trigger tU_Equipment on Equipment for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Equipment */
begin
declare @numrows int,
        @nullcnt int,
        @validcnt int,
        @insNumberEquipment int,

```

```

        @errno int,
        @errmsg varchar(255)

select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/52 Counters ON PARENT UPDATE SET
NULL */
if
    /* update(NumberEquipment) */
    update(NumberEquipment)
begin
    update Counters
    set
        /* Counters.NumberEquipment = NULL */
        Counters.NumberEquipment = NULL
    from Counters,deleted
    where
        /* Counters.NumberEquipment = deleted.NumberEquipment
*/
        Counters.NumberEquipment = deleted.NumberEquipment
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/49 Work_Log ON PARENT UPDATE SET
NULL */
if
    /* update(NumberEquipment) */
    update(NumberEquipment)
begin
    update Work_Log
    set
        /* Work_Log.NumberEquipment = NULL */
        Work_Log.NumberEquipment = NULL
    from Work_Log,deleted
    where
        /* Work_Log.NumberEquipment = delet-
ed.NumberEquipment */

```

```
Work_Log.NumberEquipment = deleted.NumberEquipment  
end
```

```
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */  
/* Equipment R/48 Schedule_of_TI ON PARENT UPDATE SET  
NULL */  
if  
  /* update(NumberEquipment) */  
  update(NumberEquipment)  
begin  
  update Schedule_of_TI  
  set  
    /* Schedule_of_TI.NumberEquipment = NULL */  
    Schedule_of_TI.NumberEquipment = NULL  
  from Schedule_of_TI,deleted  
  where  
    /* Schedule_of_TI.NumberEquipment = delet-  
ed.NumberEquipment */  
    Schedule_of_TI.NumberEquipment = delet-  
ed.NumberEquipment  
end
```

```
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */  
/* Equipment R/45 Documents ON PARENT UPDATE SET  
NULL */  
if  
  /* update(NumberEquipment) */  
  update(NumberEquipment)  
begin  
  update Documents  
  set  
    /* Documents.NumberEquipment = NULL */  
    Documents.NumberEquipment = NULL  
  from Documents,deleted  
  where  
    /* Documents.NumberEquipment = delet-  
ed.NumberEquipment */
```

```
Documents.NumberEquipment = deleted.NumberEquipment
end
```

```
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
```

```
/* Equipment R/39 Compliance ON PARENT UPDATE SET
NULL */
```

```
if
```

```
/* update(NumberEquipment) */
```

```
update(NumberEquipment)
```

```
begin
```

```
update Compliance
```

```
set
```

```
/* Compliance.NumberEquipment = NULL */
```

```
Compliance.NumberEquipment = NULL
```

```
from Compliance,deleted
```

```
where
```

```
/* Compliance.NumberEquipment = delet-
```

```
ed.NumberEquipment */
```

```
Compliance.NumberEquipment = deleted.NumberEquipment
```

```
end
```

```
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
```

```
/* Shop R/47 Equipment ON CHILD UPDATE SET NULL */
```

```
if
```

```
/* update(NumberShop) */
```

```
update(NumberShop)
```

```
begin
```

```
update Equipment
```

```
set
```

```
/* Equipment.NumberShop = NULL */
```

```
Equipment.NumberShop = NULL
```

```
from Equipment,inserted
```

```
where
```

```
/* */
```

```
and
```

```
not exists (
```

```
select * from Shop
```

```

        where
        /* inserted.NumberShop = Shop.NumberShop */
        inserted.NumberShop = Shop.NumberShop
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiseerror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tD_Materials on Materials for DELETE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Materials */
begin
    declare @errno int,
            @errmsg varchar(255)
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Materials R/50 Work_Log ON PARENT DELETE SET
    NULL */
    update Work_Log
    set
        /* Work_Log.Number_materials = NULL */
        Work_Log.Number_materials = NULL
    from Work_Log,deleted
    where
        /* Work_Log.Number_materials = deleted.Number_materials
    */
        Work_Log.Number_materials = deleted.Number_materials

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return

```

```

error:
    raiserror @errno @errmsg
    rollback transaction
end
go

create trigger tU_Materials on Materials for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Materials */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @insNumber_materials int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Materials R/50 Work_Log ON PARENT UPDATE SET
    NULL */
    if
        /* update(Number_materials) */
        update(Number_materials)
    begin
        update Work_Log
        set
            /* Work_Log.Number_materials = NULL */
            Work_Log.Number_materials = NULL
        from Work_Log,deleted
        where
            /* Work_Log.Number_materials = deleted.Number_materials
            */
            Work_Log.Number_materials = deleted.Number_materials
    end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tD_Organization on Organization for DELETE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Organization */
begin
    declare @errno int,
            @errmsg varchar(255)
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Organization R/20 Documents ON PARENT DELETE SET
    NULL */
    update Documents
    set
        /* Documents.NumberOfOrganization = NULL */
        Documents.NumberOfOrganization = NULL
    from Documents,deleted
    where
        /* Documents.NumberOfOrganization = delet-
ed.NumberOfOrganization */
        Documents.NumberOfOrganization = delet-
ed.NumberOfOrganization

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Organization R/5 Employees ON PARENT DELETE SET
NULL */
update Employees
set
    /* Employees.NumberOfOrganization = NULL */
    Employees.NumberOfOrganization = NULL
from Employees,deleted
where

```

```

/* Employees.NumberOfOrganization = delet-
ed.NumberOfOrganization */
Employees.NumberOfOrganization = delet-
ed.NumberOfOrganization

```

```

/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tU_Organization on Organization for UPDATE as
/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Organization */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @insNumberOfOrganization int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
    /* Organization R/20 Documents ON PARENT UPDATE SET
    NULL */
    if
        /* update(NumberOfOrganization) */
        update(NumberOfOrganization)
    begin
        update Documents
        set
            /* Documents.NumberOfOrganization = NULL */
            Documents.NumberOfOrganization = NULL
    end

```

```

    from Documents,deleted
    where
        /* Documents.NumberOfOrganization = delet-
ed.NumberOfOrganization */
        Documents.NumberOfOrganization = delet-
ed.NumberOfOrganization
    end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Organization R/5 Employees ON PARENT UPDATE SET
NULL */
if
    /* update(NumberOfOrganization) */
    update(NumberOfOrganization)
begin
    update Employees
    set
        /* Employees.NumberOfOrganization = NULL */
        Employees.NumberOfOrganization = NULL
    from Employees,deleted
    where
        /* Employees.NumberOfOrganization = delet-
ed.NumberOfOrganization */
        Employees.NumberOfOrganization = delet-
ed.NumberOfOrganization
    end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tD_Schedule_of_TI on Schedule_of_TI for DELETE
as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Schedule_of_TI */
begin
  declare @errno int,
          @errmsg varchar(255)
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Schedule_of_TI R/44 Documents ON PARENT DELETE
SET NULL */
  update Documents
  set
    /* Documents.Number_of_TO = NULL */
    Documents.Number_of_TO = NULL
  from Documents,deleted
  where
    /* Documents.Number_of_TO = deleted.Number_of_TO */
    Documents.Number_of_TO = deleted.Number_of_TO

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tI_Schedule_of_TI on Schedule_of_TI for INSERT
as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Schedule_of_TI */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @errno int,

```

```

        @errmsg varchar(255)

select @numrows = @@rowcount
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/48 Schedule_of_TI ON CHILD INSERT SET
NULL */
if
    /* update(NumberEquipment) */
    update(NumberEquipment)
begin
    update Schedule_of_TI
    set
        /* Schedule_of_TI.NumberEquipment = NULL */
        Schedule_of_TI.NumberEquipment = NULL
    from Schedule_of_TI,inserted
    where
        /* */
        and
        not exists (
            select * from Equipment
            where
                /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
                inserted.NumberEquipment = Equip-
ment.NumberEquipment
        )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiserror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tU_Schedule_of_TI on Schedule_of_TI for UP-
DATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Schedule_of_TI */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @insNumber_of_TO int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Schedule_of_TI R/44 Documents ON PARENT UPDATE
SET NULL */
  if
    /* update(Number_of_TO) */
    update(Number_of_TO)
  begin
    update Documents
    set
      /* Documents.Number_of_TO = NULL */
      Documents.Number_of_TO = NULL
    from Documents,deleted
    where
      /* Documents.Number_of_TO = deleted.Number_of_TO */
      Documents.Number_of_TO = deleted.Number_of_TO
  end

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Equipment R/48 Schedule_of_TI ON CHILD UPDATE SET
NULL */
  if
    /* update(NumberEquipment) */
    update(NumberEquipment)
  begin

```

```

update Schedule_of_TI
set
  /* Schedule_of_TI.NumberEquipment = NULL */
  Schedule_of_TI.NumberEquipment = NULL
from Schedule_of_TI,inserted
where
  /* */
  and
  not exists (
    select * from Equipment
    where
      /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
      inserted.NumberEquipment = Equip-
ment.NumberEquipment
  )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

```

create trigger tD_Shop on Shop for DELETE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* DELETE trigger on Shop */
begin
  declare @errno int,
    @errmsg varchar(255)
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Shop R/47 Equipment ON PARENT DELETE SET NULL */
  update Equipment
  set

```

```

/* Equipment.NumberShop = NULL */
Equipment.NumberShop = NULL
from Equipment,deleted
where
/* Equipment.NumberShop = deleted.NumberShop */
Equipment.NumberShop = deleted.NumberShop

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/43 Documents ON PARENT DELETE SET NULL */
update Documents
set
/* Documents.NumberShop = NULL */
Documents.NumberShop = NULL
from Documents,deleted
where
/* Documents.NumberShop = deleted.NumberShop */
Documents.NumberShop = deleted.NumberShop

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/38 Compliance ON PARENT DELETE SET NULL
*/
update Compliance
set
/* Compliance.NumberShop = NULL */
Compliance.NumberShop = NULL
from Compliance,deleted
where
/* Compliance.NumberShop = deleted.NumberShop */
Compliance.NumberShop = deleted.NumberShop

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
raiserror @errno @errmsg
rollback transaction
end

```

go

```
create trigger tU_Shop on Shop for UPDATE as
/* ERwin Bultin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Shop */
begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @insNumberShop int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
  /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
  /* Shop R/47 Equipment ON PARENT UPDATE SET NULL */
  if
    /* update(NumberShop) */
    update(NumberShop)
  begin
    update Equipment
    set
      /* Equipment.NumberShop = NULL */
      Equipment.NumberShop = NULL
    from Equipment,deleted
    where
      /* Equipment.NumberShop = deleted.NumberShop */
      Equipment.NumberShop = deleted.NumberShop
  end

  /* ERwin Bultin Sat Apr 20 22:19:41 2019 */
  /* Shop R/43 Documents ON PARENT UPDATE SET NULL */
  if
    /* update(NumberShop) */
    update(NumberShop)
  begin
    update Documents
```

```

set
/* Documents.NumberShop = NULL */
Documents.NumberShop = NULL
from Documents,deleted
where
/* Documents.NumberShop = deleted.NumberShop */
Documents.NumberShop = deleted.NumberShop
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Shop R/38 Compliance ON PARENT UPDATE SET NULL */
if
/* update(NumberShop) */
update(NumberShop)
begin
update Compliance
set
/* Compliance.NumberShop = NULL */
Compliance.NumberShop = NULL
from Compliance,deleted
where
/* Compliance.NumberShop = deleted.NumberShop */
Compliance.NumberShop = deleted.NumberShop
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiseerror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tI_Work_Log on Work_Log for INSERT as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* INSERT trigger on Work_Log */

```

```

begin
  declare @numrows int,
          @nullcnt int,
          @validcnt int,
          @errno int,
          @errmsg varchar(255)

  select @numrows = @@rowcount
  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Employees R/51 Work_Log ON CHILD INSERT SET NULL
  */
  if
    /* update(nomerOfEmployees) */
    update(nomerOfEmployees)
  begin
    update Work_Log
    set
      /* Work_Log.nomerOfEmployees = NULL */
      Work_Log.nomerOfEmployees = NULL
    from Work_Log,inserted
    where
      /* */
      and
      not exists (
        select * from Employees
        where
          /* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
          inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees
      )
  end

  /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
  /* Materials R/50 Work_Log ON CHILD INSERT SET NULL */
  if
    /* update(Number_materials) */

```

```

    update(Number_materials)
begin
    update Work_Log
    set
        /* Work_Log.Number_materials = NULL */
        Work_Log.Number_materials = NULL
    from Work_Log,inserted
    where
        /* */
        and
        not exists (
            select * from Materials
            where
                /* inserted.Number_materials = Materi-
als.Number_materials */
                inserted.Number_materials = Materials.Number_materials
        )
end

```

```

/* ERwin Built-in Sat Apr 20 22:19:41 2019 */
/* Equipment R/49 Work_Log ON CHILD INSERT SET NULL
*/
if
    /* update(NumberEquipment) */
    update(NumberEquipment)
begin
    update Work_Log
    set
        /* Work_Log.NumberEquipment = NULL */
        Work_Log.NumberEquipment = NULL
    from Work_Log,inserted
    where
        /* */
        and
        not exists (
            select * from Equipment
            where

```

```

        /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
        inserted.NumberEquipment = Equip-
ment.NumberEquipment
    )
end

```

```

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
    raiseerror @errno @errmsg
    rollback transaction
end
go

```

```

create trigger tU_Work_Log on Work_Log for UPDATE as
/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* UPDATE trigger on Work_Log */
begin
    declare @numrows int,
            @nullcnt int,
            @validcnt int,
            @insNumber_Work_Log int,
            @errno int,
            @errmsg varchar(255)

    select @numrows = @@rowcount
    /* ERwin Builtin Sat Apr 20 22:19:41 2019 */
    /* Employees R/51 Work_Log ON CHILD UPDATE SET
NULL */
    if
        /* update(nomerOfEmployees) */
        update(nomerOfEmployees)
    begin
        update Work_Log
        set

```

```

/* Work_Log.nomerOfEmployees = NULL */
Work_Log.nomerOfEmployees = NULL
from Work_Log,inserted
where
/* */
and
not exists (
select * from Employees
where
/* inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees */
inserted.nomerOfEmployees = Employ-
ees.nomerOfEmployees
)
end

/* ERwin Built in Sat Apr 20 22:19:41 2019 */
/* Materials R/50 Work_Log ON CHILD UPDATE SET NULL
*/
if
/* update(Number_materials) */
update(Number_materials)
begin
update Work_Log
set
/* Work_Log.Number_materials = NULL */
Work_Log.Number_materials = NULL
from Work_Log,inserted
where
/* */
and
not exists (
select * from Materials
where
/* inserted.Number_materials = Materi-
als.Number_materials */
inserted.Number_materials = Materials.Number_materials

```

```

    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
/* Equipment R/49 Work_Log ON CHILD UPDATE SET NULL */
if
  /* update(NumberEquipment) */
  update(NumberEquipment)
begin
  update Work_Log
  set
    /* Work_Log.NumberEquipment = NULL */
    Work_Log.NumberEquipment = NULL
  from Work_Log,inserted
  where
    /* */
    and
    not exists (
      select * from Equipment
      where
        /* inserted.NumberEquipment = Equip-
ment.NumberEquipment */
        inserted.NumberEquipment = Equip-
ment.NumberEquipment
    )
end

/* ERwin Builtin Sat Apr 20 22:19:41 2019 */
return
error:
  raiserror @errno @errmsg
  rollback transaction
end
go

```

## ПРИЛОЖЕНИЕ 2

### Пример логов СУБД PostgreSQL

2019-04-14 12:04:18.934 MSK [14984] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:06:14.744 MSK [9568] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:06:14.744 MSK [9568] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:06:14.744 MSK [1516] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:06:14.744 MSK [1516] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:06:14.744 MSK [13996] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:06:14.745 MSK [13620] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:06:14.745 MSK [13620] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:12:54.548 MSK [13852] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:12:54.549 MSK [13852] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:12:54.549 MSK [11200] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:13:47.377 MSK [3932] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:13:47.377 MSK [3188] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:13:47.377 MSK [3932] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:16:07.694 MSK [2988] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:16:07.694 MSK [2988] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:16:07.695 MSK [5148] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:16:07.695 MSK [5796] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:16:07.695 MSK [5796] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:17:07.539 MSK [12068] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:17:07.539 MSK [12068] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:17:07.539 MSK [15024] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:17:07.539 MSK [15024] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:17:07.540 MSK [6812] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:17:07.540 MSK [6812] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:17:07.540 MSK [10672] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:19:43.866 MSK [11200] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:19:43.866 MSK [11200] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:19:43.866 MSK [3268] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:19:43.866 MSK [3268] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:19:43.866 MSK [2388] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:19:43.866 MSK [2388] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:19:43.867 MSK [12052] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:21:36.889 MSK [10092] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:21:36.889 MSK [10092] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:21:36.889 MSK [10664] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:24:28.269 MSK [9896] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:24:28.269 MSK [9896] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:24:28.269 MSK [11960] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:24:28.269 MSK [5044] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:24:28.269 MSK [11960] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:26:01.972 MSK [4876] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:26:01.972 MSK [4876] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:26:01.972 MSK [13340] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:26:01.972 MSK [14452] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:26:01.972 MSK [14452] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:30:18.696 MSK [11352] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:30:18.696 MSK [7468] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:30:18.696 MSK [4592] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:30:18.696 MSK [4592] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:30:18.696 MSK [11352] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:39:45.024 MSK [15304] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:39:45.024 MSK [15304] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:39:45.024 MSK [14428] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:40:11.225 MSK [5980] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:40:57.169 MSK [1508] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:40:57.169 MSK [1508] СООБЩЕНИЕ: неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:40:57.169 MSK [3376] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:50:57.193 MSK [13968] СООБЩЕНИЕ: не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:50:57.193 MSK [13968] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:50:57.193 MSK [14308] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:50:57.193 MSK [14308] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:50:57.193 MSK [14352] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:50:57.193 MSK [616] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:50:57.193 MSK [616] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:53:35.425 MSK [10660] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:53:35.425 MSK [10660] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:53:35.425 MSK [8452] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:53:35.425 MSK [12832] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:53:35.425 MSK [12832] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой транзакции

2019-04-14 12:53:35.425 MSK [14524] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection was forcibly closed by the remote host.

2019-04-14 12:53:35.425 MSK [14524] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:54:33.066 MSK [11588] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:54:33.066 MSK [2688] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:54:33.066 MSK [8084] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:54:33.066 MSK [2688] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:54:33.066 MSK [8084] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:55:47.586 MSK [14228] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:55:47.586 MSK [14228] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:55:47.586 MSK [12016] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:55:59.284 MSK [14648] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:56:57.910 MSK [8312] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:56:57.910 MSK [8312] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:56:57.910 MSK [14760] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 12:56:57.910 MSK [14760] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 12:56:57.910 MSK [8476] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:57:46.205 MSK [5404] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:57:46.205 MSK [6676] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 12:57:46.205 MSK [6676] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:00:21.664 MSK [1480] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 13:00:21.664 MSK [1480] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:00:21.664 MSK [10840] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 13:00:21.664 MSK [2640] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 13:00:21.664 MSK [2640] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:01:53.167 MSK [14456] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 13:01:53.167 MSK [13628] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 13:01:53.167 MSK [13628] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:01:53.167 MSK [13624] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 13:01:53.167 MSK [13624] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:03:52.578 MSK [13528] СООБЩЕНИЕ:  
не удалось получить данные от клиента: An existing connection  
was forcibly closed by the remote host.

2019-04-14 13:03:52.578 MSK [13528] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:03:52.579 MSK [7372] СООБЩЕНИЕ: не  
удалось получить данные от клиента: An existing connec-tion  
was forcibly closed by the remote host.

2019-04-14 13:03:52.579 MSK [7372] СООБЩЕНИЕ:  
неожиданный обрыв соединения с клиентом при открытой  
транзакции

2019-04-14 13:03:52.579 MSK [464] СООБЩЕНИЕ: не удалось  
получить данные от клиента: An existing connec-tion was forci-  
bly closed by the remote host.

## ПРИЛОЖЕНИЕ 3

### Пример логов СУБД MySQL

Дата,Источник,Серьезность,Сообщение

04/23/2019 22:11:33,spid53,Неизвестно,Using 'xpstar.dll' version '2017.140.1000' to execute extended stored procedure 'xp\_enumerrorlogs'. This is an informational message only; no user action is required.

04/23/2019 22:11:33,spid53,Неизвестно,Attempting to load library 'xpstar.dll' into memory. This is an informational message only. No user action is required.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:35,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:34,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:33,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:32,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:32,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:32,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:32,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'new123' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'new123' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Starting up database 'new123'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'qwe' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'qwe' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Starting up database 'qwe'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'KursModeling' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'KursModeling' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Starting up database 'KursModeling'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'Student' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'Student' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Starting up database 'Student'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'lab 5' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'lab 5' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Starting up database 'lab 5'.

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is shutdown for database 'lab4' with worker pool size [2].

04/22/2019 10:57:22,spid51,Неизвестно,Parallel redo is started for database 'lab4' with worker pool size [2].

04/22/2019 10:57:21,spid51,Неизвестно,Starting up database 'lab4'.

04/22/2019 10:57:21,spid51,Неизвестно,Parallel redo is shutdown for database 'arenda' with worker pool size [2].

04/22/2019 10:57:21,spid51,Неизвестно,Parallel redo is started for database 'arenda' with worker pool size [2].

04/22/2019 10:57:21,spid51,Неизвестно,Starting up database 'arenda'.

04/22/2019 10:57:21,spid51,Неизвестно,Parallel redo is shutdown for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:21,spid51,Неизвестно,Parallel redo is started for database 'Vhod' with worker pool size [2].

04/22/2019 10:57:21,spid51,Неизвестно,Starting up database 'Vhod'.

04/22/2019 10:57:21,spid51,Неизвестно,Using 'x-plog70.dll' version '2017.140.1000' to execute extended stored

procedure 'xp\_msver'. This is an informational message only; no user action is required.

04/22/2019 10:57:21,spid51,Неизвестно,Attempting to load library 'xplog70.dll' into memory. This is an informational message only. No user action is required.

04/22/2019 10:52:27,spid5s,Неизвестно,Recovery is complete. This is an informational message only. No user action is required.

04/22/2019 10:52:27,spid19s,Неизвестно,Service Broker manager has started.

04/22/2019 10:52:27,spid19s,Неизвестно,The Database Mirroring endpoint is in disabled or stopped state.

04/22/2019 10:52:27,spid19s,Неизвестно,The Service Broker endpoint is in disabled or stopped state.

04/22/2019 10:52:27,spid12s,Неизвестно,Starting up database 'tempdb'.

04/22/2019 10:52:26,spid12s,Неизвестно,Clearing tempdb database.

04/22/2019 10:52:26,spid12s,Неизвестно,Polybase feature disabled.

04/22/2019 10:52:26,spid12s,Неизвестно,Starting up database 'model'.

04/22/2019 10:52:26,spid12s,Неизвестно,The resource database build version is 14.00.1000. This is an informational message only. No user action is required.

04/22/2019 10:52:26,spid12s,Неизвестно,Starting up database 'mssqlsystemresource'.

04/22/2019 10:52:26,spid5s,Неизвестно,Starting up database 'msdb'.

04/22/2019 10:52:26,spid16s,Неизвестно,A new instance of the full-text filter daemon host process has been successfully started.

04/22/2019 10:52:25,Server,Неизвестно,The SQL Server Network Interface library could not register the Service Principal Name (SPN) [ MSSQLSvc/DESKTOP-656BS6R:1234 ] for the SQL Server service. Windows return code: 0xffffffff<c/> state: 63. Failure to register a SPN might cause integrated authentication to

use NTLM instead of Kerberos. This is an informational message. Further action is only required if Kerberos authentication is required by authentication policies and if the SPN has not been manually registered.

04/22/2019 10:52:25,Server,Неизвестно,The SQL Server Network Interface library could not register the Service Principal Name (SPN) [ MSSQLSvc/DESKTOP-656BS6R:SQLEXPRESS ] for the SQL Server service. Windows return code: 0xffffffff<c/> state: 63. Failure to register a SPN might cause integrated authentication to use NTLM instead of Kerberos. This is an informational message. Further action is only required if Kerberos authentication is required by authentication policies and if the SPN has not been manually registered.

04/22/2019 10:52:25,spid15s,Неизвестно,SQL Server is now ready for client connections. This is an informational message; no user action is required.

04/22/2019 10:52:25,Server,Неизвестно,SQL Server is attempting to register a Service Principal Name (SPN) for the SQL Server service. Kerberos authentication will not be possible until a SPN is registered for the SQL Server service. This is an informational message. No user action is required.

04/22/2019 10:52:25,spid15s,Неизвестно,Dedicated administrator connection support was not started because it is disabled on this edition of SQL Server. If you want to use a dedicated administrator connection<c/> restart SQL Server using the trace flag 7806. This is an informational message only. No user action is required.

04/22/2019 10:52:25,spid15s,Неизвестно,Server local connection provider is ready to accept connection on [ \\.\pipe\MSSQL\$SQLEXPRESS\sql\query ].

04/22/2019 10:52:25,spid15s,Неизвестно,Server local connection provider is ready to accept connection on [ \\.\pipe\SQLLocal\SQLEXPRESS ].

04/22/2019 10:52:25,spid15s,Неизвестно,Server is listening on [ 'any' <ipv4> 49712 ].

04/22/2019 10:52:25,spid15s,Неизвестно,Server is listening on [ 'any' <ipv6> 49712 ].

04/22/2019 10:52:25,spid15s,Неизвестно,Server is listening on [ 'any' <ipv4> 1234].

04/22/2019 10:52:25,spid15s,Неизвестно,Server is listening on [ 'any' <ipv6> 1234].

04/22/2019 10:52:25,spid15s,Неизвестно,A self-generated certificate was successfully loaded for encryption.

04/22/2019 10:52:25,spid5s,Неизвестно,Server name is 'DESKTOP-656BS6R\SQLEXPRESS'. This is an informational message only. No user action is required.

04/22/2019 10:52:25,spid5s,Неизвестно,SQL Trace ID 1 was started by login "sa".

04/22/2019 10:52:25,spid5s,Неизвестно,SQL Server Audit has started the audits. This is an informational message. No user action is required.

04/22/2019 10:52:25,spid5s,Неизвестно,SQL Server Audit is starting the audits. This is an informational message. No user action is required.

04/22/2019 10:52:25,Server,Неизвестно,Common language runtime (CLR) functionality initialized using CLR version v4.0.30319 from C:\Windows\Microsoft.NET\Framework64\v4.0.30319\.

04/22/2019 10:52:25,spid5s,Неизвестно,Starting up database 'master'.

04/22/2019 10:52:25,Server,Неизвестно,Software Usage Metrics is disabled.

04/22/2019 10:52:25,Server,Неизвестно,Query Store settings initialized with enabled = 1<c/>

04/22/2019 10:52:25,Server,Неизвестно,Database Instant File Initialization: отключено. For security and performance considerations see the topic 'Database Instant File Initialization' in SQL Server Books Online. This is an informational message only. No user action is required.

04/22/2019 10:52:24,Server,Неизвестно,CLR version v4.0.30319 loaded.

04/22/2019 10:52:24,Server,Неизвестно,In-Memory OLTP initialized on lowend machine.

04/22/2019 10:52:24,Server,Неизвестно,Using dynamic lock allocation. Initial allocation of 2500 Lock blocks and 5000 Lock Owner blocks per node. This is an informational message only. No user action is required.

04/22/2019 10:52:24,Server,Неизвестно,Node configuration: node 0: CPU mask: 0x000000000000000f:0 Active CPU mask: 0x000000000000000f:0. This message provides a description of the NUMA configuration for this computer. This is an informational message only. No user action is required.

04/22/2019 10:52:24,Server,Неизвестно,This instance of SQL Server last reported using a process ID of 5884 at 21.04.2019 23:17:31 (local) 21.04.2019 20:17:31 (UTC). This is an informational message only; no user action is required.

04/22/2019 10:52:23,Server,Неизвестно,The maximum number of dedicated administrator connections for this instance is '1'

04/22/2019 10:52:23,Server,Неизвестно,Implied authentication manager initialization failed. Implied authentication will be disabled.

04/22/2019 10:52:23,Server,Неизвестно,InitializeExternalUserGroupSid failed. Implied authentication will be disabled.

04/22/2019 10:52:23,Server,Неизвестно,Perfmon counters for resource governor pools and groups failed to initialize and are disabled.

04/22/2019 10:52:22,Server,Неизвестно,Buffer pool extension is already disabled. No action is necessary.

04/22/2019 10:52:21,Server,Неизвестно,Default collation: Cyrillic\_General\_CI\_AS (русский 1049)

04/22/2019 10:52:21,Server,Неизвестно,Cannot query value 'First Counter' associated with registry key 'HKLM\SYSTEM\CurrentControlSet\Services\MSSQL\$SQLEXPRESS\Performance'. SQL Server performance counters are disabled.

04/22/2019 10:52:21,Server,Неизвестно,Ошибка: 8317<c/> серьезность: 16<c/> состояние: 1.

04/22/2019 10:52:20,Server,Неизвестно,Using conventional memory in the memory manager.

04/22/2019 10:52:20,Server,Неизвестно,Detected 8075 MB of RAM. This is an informational message; no user action is required.

04/22/2019 10:52:20,Server,Неизвестно,SQL Server is starting at normal priority base (=7). This is an informational message only. No user action is required.

04/22/2019 10:52:20,Server,Неизвестно,SQL Server detected 1 sockets with 4 cores per socket and 4 logical processors per socket<c/> 4 total logical processors; using 4 logical processors based on SQL Server licensing. This is an informational message; no user action is required.

04/22/2019 10:52:18,Server,Неизвестно,Command Line Startup Parameters:<nl/> -s "SQLEXPRESS"

04/22/2019 10:52:18,Server,Неизвестно,Registry startup parameters: <nl/> -d C:\Program Files\Microsoft SQL Server\MSSQL14.SQLEXPRESS\MSSQL\DATA\master.mdf<nl/> -e C:\Program Files\Microsoft SQL Server\MSSQL14.SQLEXPRESS\MSSQL\Log\ERRORLOG<nl/> -l C:\Program Files\Microsoft SQL Server\MSSQL14.SQLEXPRESS\MSSQL\DATA\mastlog.ldf

04/22/2019 10:52:18,Server,Неизвестно,The service account is 'NT Service\MSSQL\$SQLEXPRESS'. This is an informational message; no user action is required.

04/22/2019 10:52:18,Server,Неизвестно,Logging SQL Server messages in file 'C:\Program Files\Microsoft SQL Server\MSSQL14.SQLEXPRESS\MSSQL\Log\ERRORLOG'.

04/22/2019 10:52:18,Server,Неизвестно,Authentication mode is MIXED.

04/22/2019 10:52:18,Server,Неизвестно,System Manufacturer: 'Acer'<c/> System Model: 'Aspire A715-71G'.

04/22/2019 10:52:18,Server,Неизвестно,Server process ID is 5464.

04/22/2019 10:52:18,Server,Неизвестно,All rights reserved.

04/22/2019 10:52:18,Server,Неизвестно,(c) Microsoft Corporation.

04/22/2019 10:52:18,Server,Неизвестно,UTC adjustment:  
3:00

04/22/2019 10:52:18,Server,Неизвестно,Microsoft SQL Server  
2017 (RTM) - 14.0.1000.169 (X64) <nl/> Aug 22 2017  
17:04:49 <nl/> Copyright (C) 2017 Microsoft Corporation<nl/>  
Express Edition (64-bit) on Windows 10 Enterprise 10.0  
<X64> (Build 17134: )

## ОГЛАВЛЕНИЕ

|                                                                  |     |
|------------------------------------------------------------------|-----|
| ВВЕДЕНИЕ.....                                                    | 3   |
| 1. КОМПОНЕНТЫ СУБД MYSQL.....                                    | 5   |
| 2. БЕЗОПАСНОСТЬ MYSQL.....                                       | 7   |
| 3. ПОПУЛЯРНЫЕ УТИЛИТЫ И СРЕДСТВА<br>АДМИНИСТРИРОВАНИЯ MYSQL..... | 13  |
| 3.1. MySQL Workbench.....                                        | 13  |
| 3.2. MySQL RHPMyAdmin.....                                       | 13  |
| 3.3. Утилиты мониторинга.....                                    | 14  |
| 3.4. Файл параметров MY.INI.....                                 | 16  |
| 4. ТИПЫ ТАБЛИЦ MYSQL.....                                        | 18  |
| 5. УПРАВЛЕНИЕ УЧЕТНЫМИ ЗАПИСЯМИ<br>ПОЛЬЗОВАТЕЛЕЙ MYSQL.....      | 23  |
| 6. ПРЕДОТВРАЩЕНИЕ СБОЕВ И ВОССТАНОВЛЕНИЕ.....                    | 27  |
| 7. КОМАНДЫ АДМИНИСТРИРОВАНИЯ БАЗ<br>ДАННЫХ.....                  | 30  |
| 8. ФАЙЛЫ ЖУРНАЛОВ MYSQL.....                                     | 36  |
| 9. РЕПЛИКАЦИЯ В MYSQL.....                                       | 38  |
| 10. ЛАБОРАТОРНЫЕ РАБОТЫ.....                                     | 43  |
| ЗАКЛЮЧЕНИЕ.....                                                  | 73  |
| БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....                                    | 74  |
| Приложение 1.....                                                | 75  |
| Приложение 2.....                                                | 133 |
| Приложение 3.....                                                | 142 |

Учебное издание

**Королев Евгений Николаевич**  
**Тишуков Борис Николаевич**  
**Мандрыкин Андрей Владимирович**

## **АДМИНИСТРИРОВАНИЕ СУБД**

Учебное пособие

В авторской редакции

Компьютерная верстка Е. Н. Королева

Подписано в печать 23.05.2021.

Формат 60х84/16. Бумага для множительных аппаратов.

Уч.-изд. л. 9,8. Усл. печ. л. 9,1. Тираж 350 экз.

Зак. № 77

ФГБОУ ВО «Воронежский государственный технический  
университет»

394026 Воронеж, Московский просп., 14

Участок оперативной полиграфии издательства ВГТУ

394026 Воронеж, Московский просп., 14